

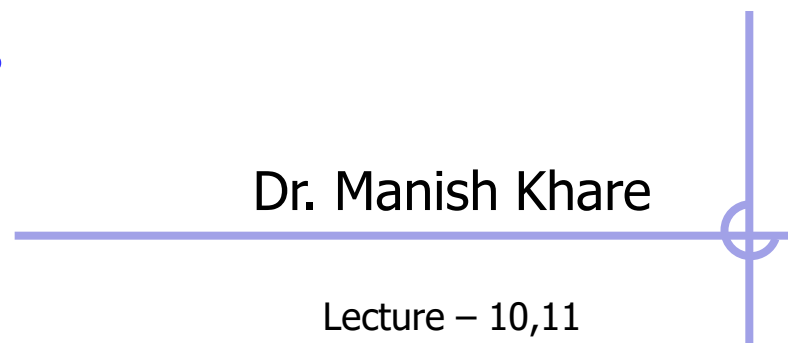
Hash Tables
Trees
Sets
Stack
Linked Lists
Graphs
Sets
Queues
DATA STRUCTURES
Linked Lists
Queues
Hash Tables
Sets
Array
Stack
Queues
Trees
Linked Lists
Graphs
Sets
Array

Data Structures

IT623

Dr. Manish Khare

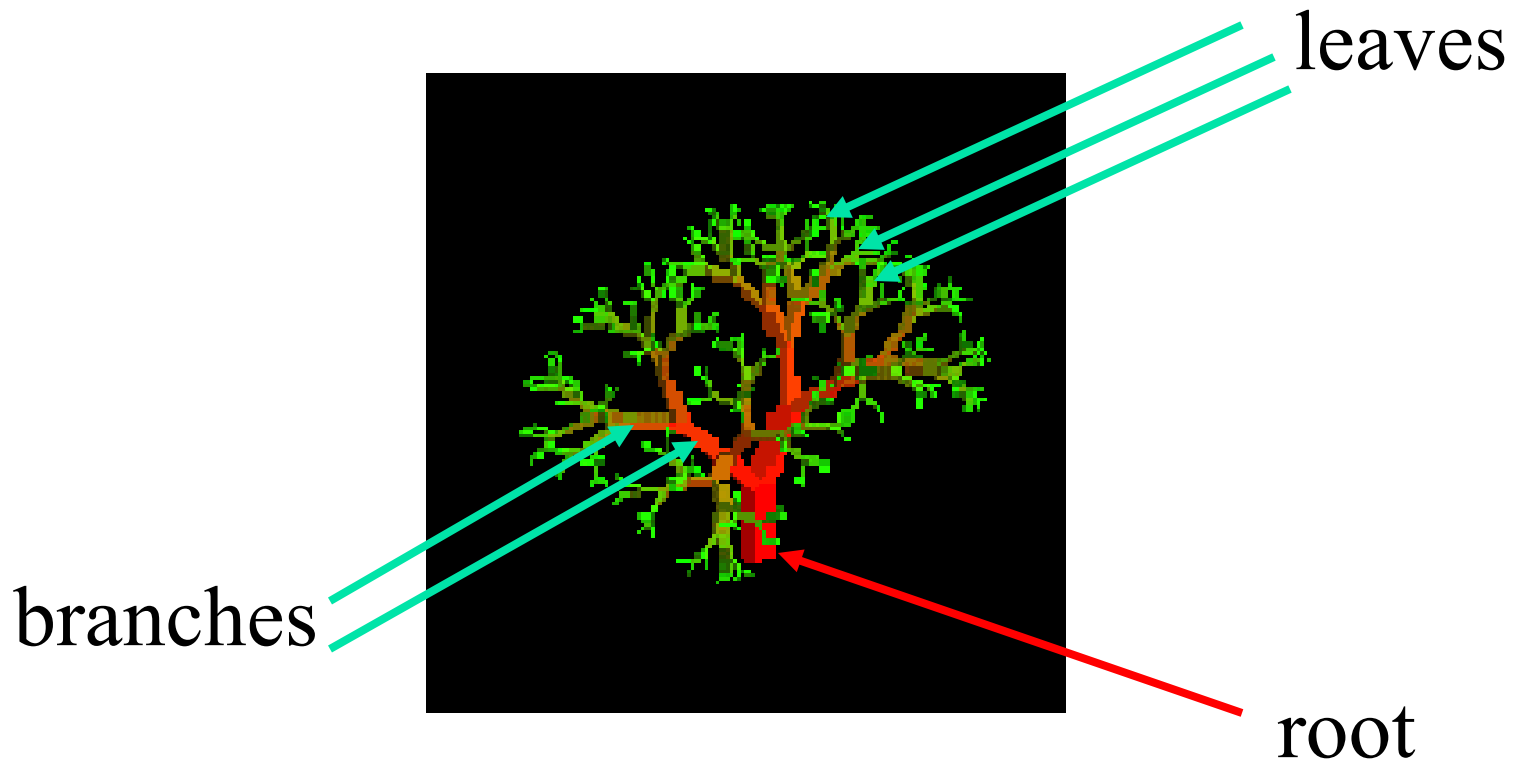
Lecture – 10,11



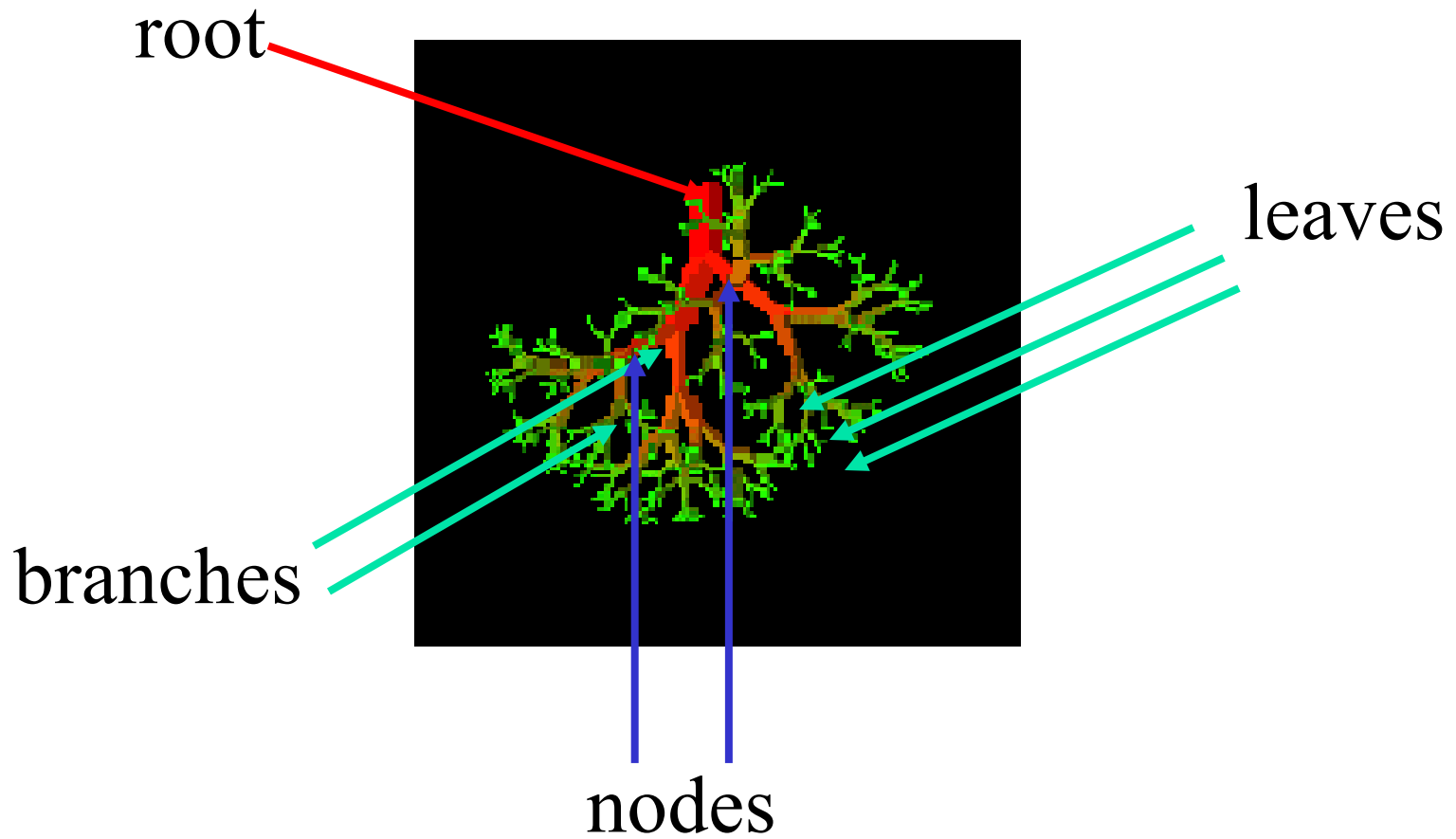


Tree

Nature View of a Tree



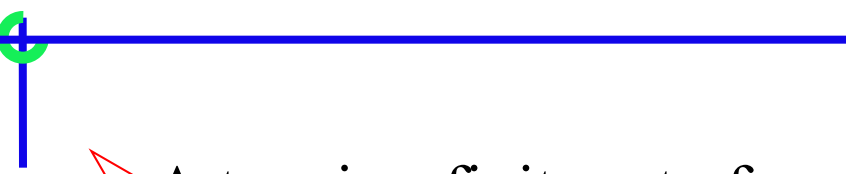
Computer Scientist's View



Tree

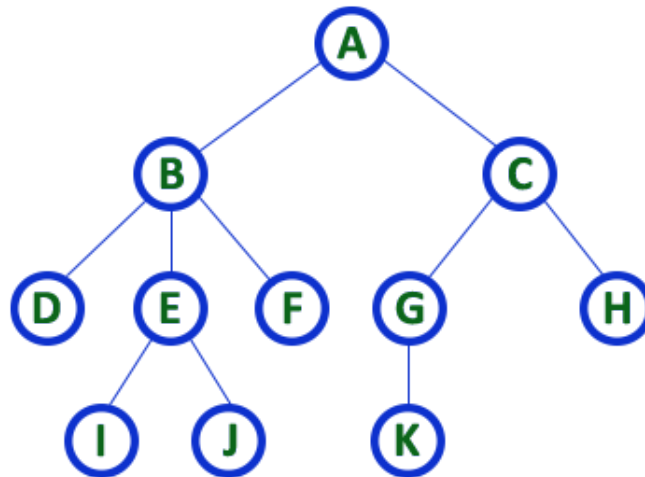
- In linear data structure, data is organized in sequential order and in non-linear data structure, data is organized in random order.
- Tree is a very popular data structure used in wide range of applications. A tree data structure can be defined as follows...
- **Tree is a non-linear data structure which organizes data in hierarchical structure and this is a recursive definition.**
- **Tree data structure is a collection of data (Node) which is organized in hierarchical structure and this is a recursive definition**

Definition of Tree

- 
- A tree is a finite set of one or more nodes such that:
 - There is a specially designated node called the root.
 - The remaining nodes are partitioned into $n \geq 0$ disjoint sets $T_1, \dots, T_n$, where each of these sets is a tree. We call $T_1, \dots, T_n$ the subtrees of the root.

Tree Terminology

- In tree data structure, every individual element is called as **Node**. Node in a tree data structure, stores the actual data of that particular element and link to next element in hierarchical structure.
- In a tree data structure, if we have **N** number of nodes then we can have a maximum of **N-1** number of links.



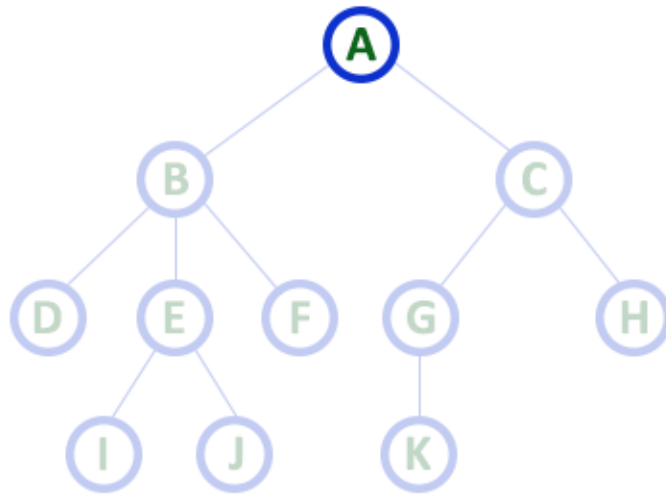
TREE with 11 nodes and 10 edges

- In any tree with '**N**' nodes there will be maximum of '**N-1**' edges
- In a tree every individual element is called as '**NODE**'

Tree Terminology

➤ Root

In a tree data structure, the first node is called as **Root Node**. Every tree must have root node. We can say that root node is the origin of tree data structure. In any tree, there must be only one root node. We never have multiple root nodes in a tree.



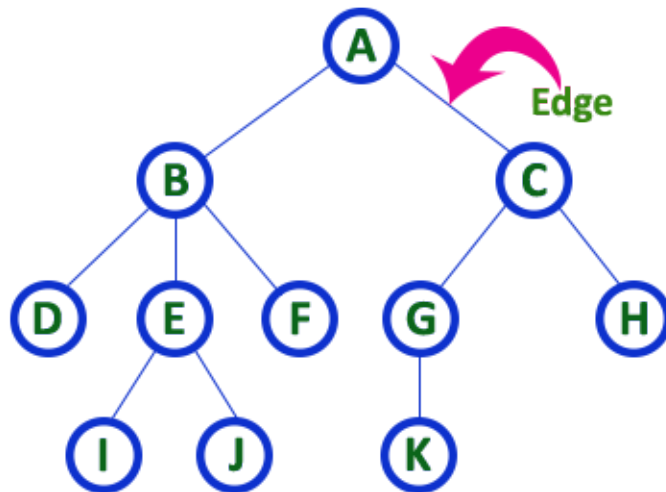
Here 'A' is the 'root' node

- In any tree the first node is called as **ROOT** node

Tree Terminology

➤ Edge

In a tree data structure, the connecting link between any two nodes is called as **EDGE**. In a tree with '**N**' number of nodes there will be a maximum of '**N-1**' number of edges.

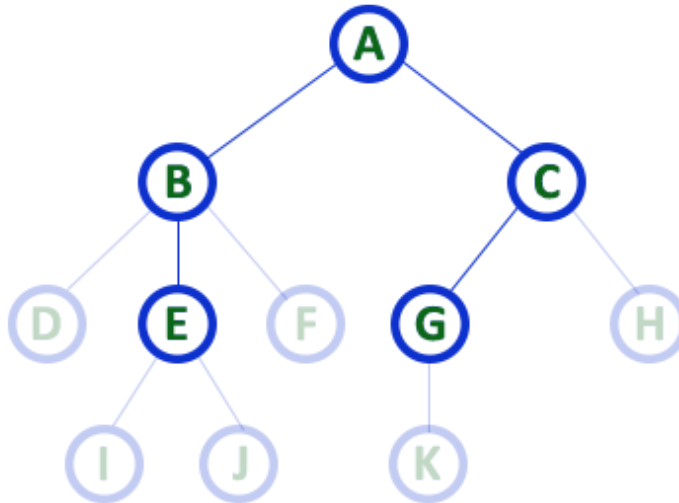


- In any tree, 'Edge' is a connecting link between two nodes.

Tree Terminology

➤ Parent

In a tree data structure, the node which is predecessor of any node is called as **PARENT NODE**. In simple words, the node which has branch from it to any other node is called as parent node. Parent node can also be defined as "**The node which has child / children**".



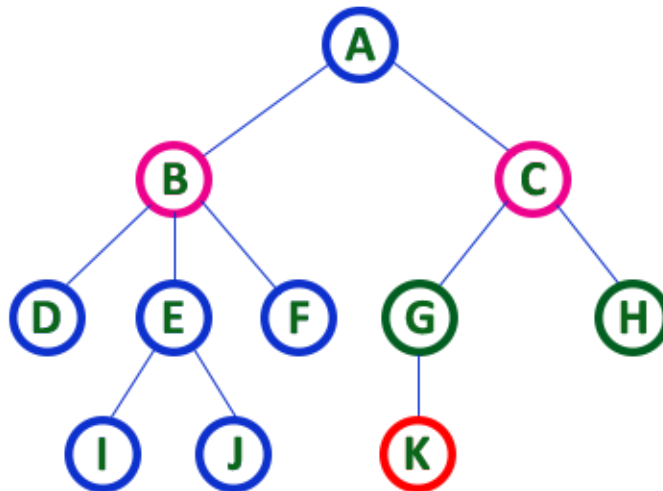
Here A, B, C, E & G are **Parent** nodes

- In any tree the node which has child / children is called '**Parent**'
- A node which is predecessor of any other node is called '**Parent**'

Tree Terminology

➤ Child

In a tree data structure, the node which is descendant of any node is called as **CHILD Node**. In simple words, the node which has a link from its parent node is called as child node. In a tree, any parent node can have any number of child nodes. In a tree, all the nodes except root are child nodes.



Here **B & C** are **Children of A**

Here **G & H** are **Children of C**

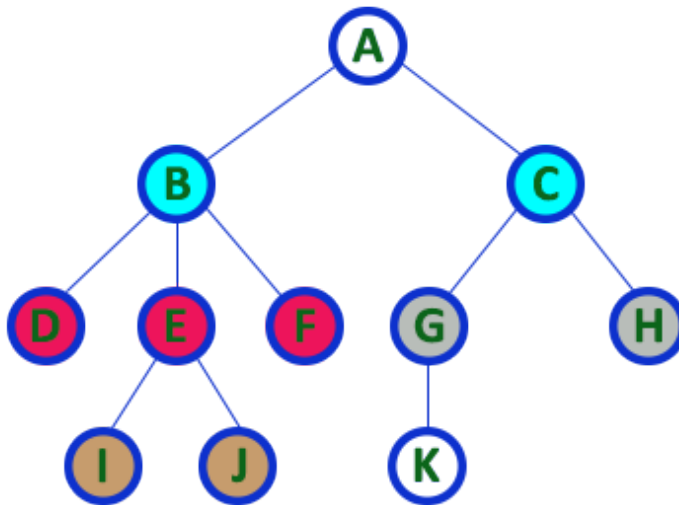
Here **K** is **Child of G**

- descendant of any node is called as **CHILD Node**

Tree Terminology

➤ Siblings

In a tree data structure, nodes which belong to same Parent are called as **SIBLINGS**. In simple words, the nodes with same parent are called as Sibling nodes.



Here **B & C** are **Siblings**

Here **D E & F** are **Siblings**

Here **G & H** are **Siblings**

Here **I & J** are **Siblings**

- In any tree the nodes which has same Parent are called '**Siblings**'

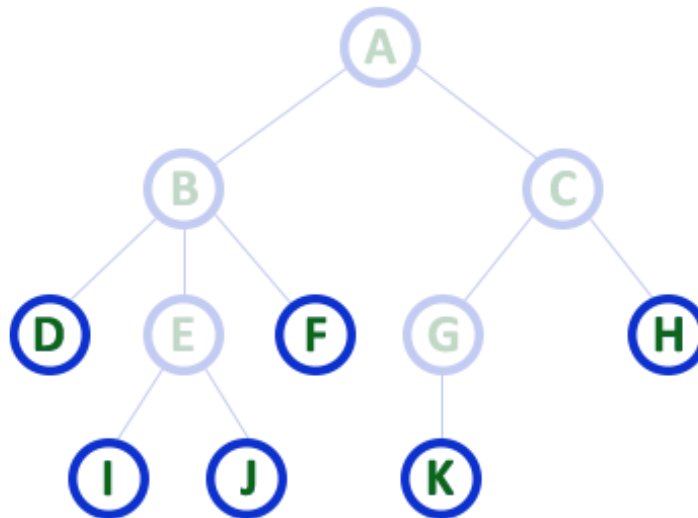
- The children of a Parent are called '**Siblings**'

Tree Terminology

➤ Leaf

In a tree data structure, the node which does not have a child is called as **LEAF Node**. In simple words, a leaf is a node with no child.

In a tree data structure, the leaf nodes are also called as **External Nodes**. External node is also a node with no child. In a tree, leaf node is also called as 'Terminal' node.



Here D, I, J, F, K & H are **Leaf** nodes

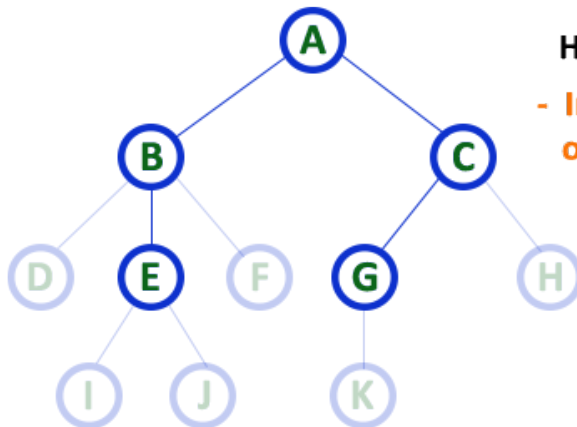
- In any tree the node which does not have children is called '**Leaf**'
- A node without successors is called a '**leaf**' node

Tree Terminology

➤ Internal Nodes

In a tree data structure, the node which has at least one child is called as **INTERNAL Node**. In simple words, an internal node is a node with at least one child.

In a tree data structure, nodes other than leaf nodes are called as **Internal Nodes**. The root node is also said to be Internal Node if the tree has more than one node. Internal nodes are also called as 'Non-Terminal' nodes.



Here A, B, C, E & G are **Internal** nodes

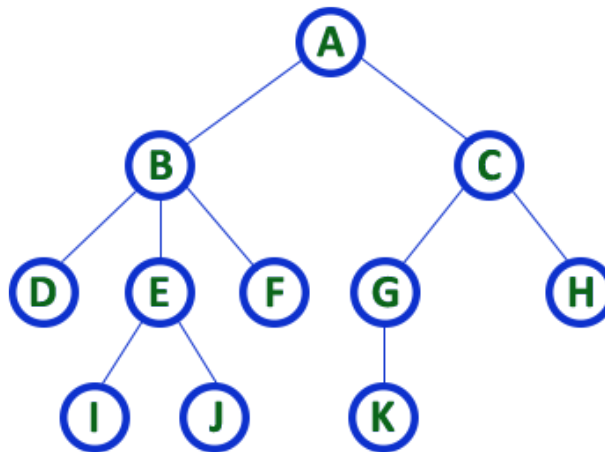
- In any tree the node which has at least one child is called '**Internal**' node

- Every non-leaf node is called as '**Internal**' node

Tree Terminology

➤ Degree

In a tree data structure, the total number of children of a node is called as **DEGREE** of that Node. In simple words, the Degree of a node is total number of children it has. The highest degree of a node among all the nodes in a tree is called as '**Degree of Tree**'



Here Degree of B is 3

Here Degree of A is 2

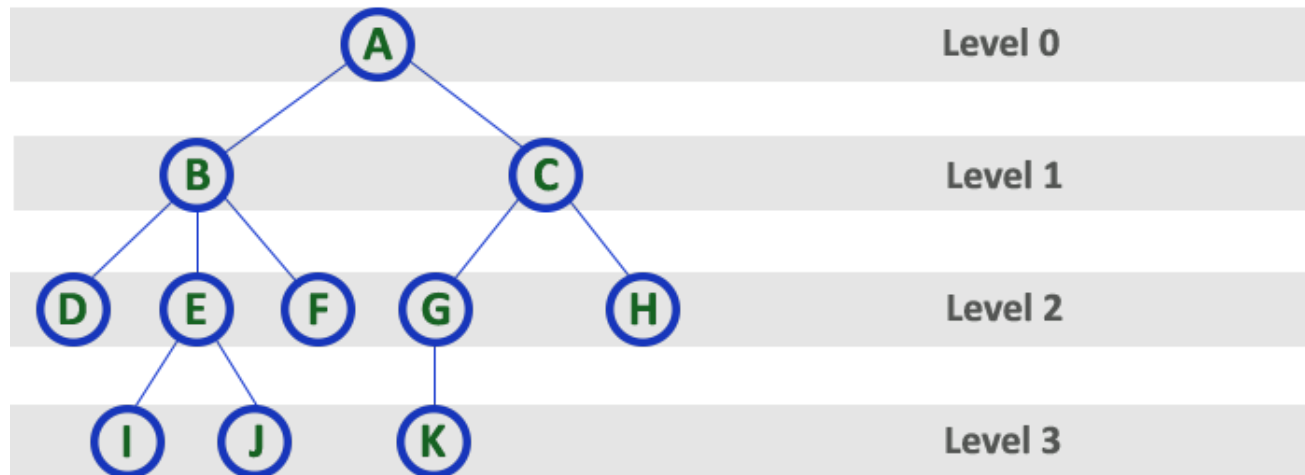
Here Degree of F is 0

- In any tree, 'Degree' a node is total number of children it has.

Tree Terminology

➤ Level

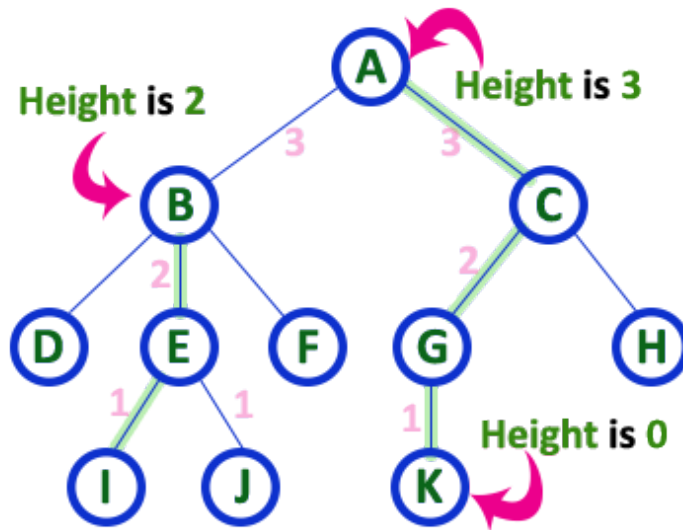
In a tree data structure, the root node is said to be at Level 0 and the children of root node are at Level 1 and the children of the nodes which are at Level 1 will be at Level 2 and so on... In simple words, in a tree each step from top to bottom is called as a Level and the Level count starts with '0' and incremented by one at each level (Step).



Tree Terminology

➤ Height

In a tree data structure, the total number of edges from leaf node to a particular node in the longest path is called as **HEIGHT** of that Node. In a tree, height of the root node is said to be **height of the tree**. In a tree, **height of all leaf nodes is '0'.**



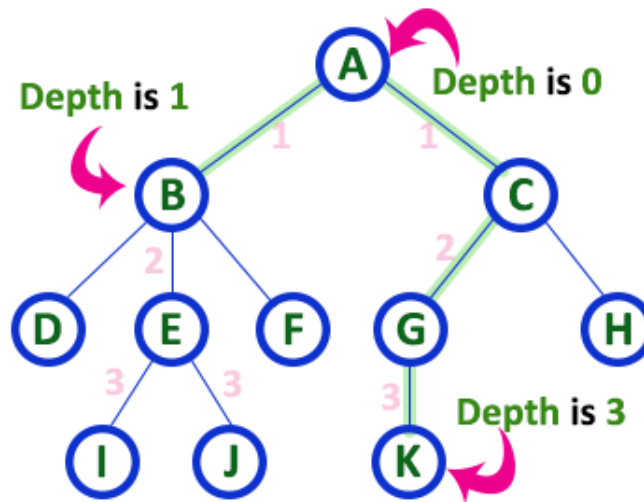
Here Height of tree is 3

- In any tree, 'Height of Node' is total number of Edges from leaf to that node in longest path.
- In any tree, 'Height of Tree' is the height of the root node.

Tree Terminology

➤ Depth

In a tree data structure, the total number of edges from root node to a particular node is called as **DEPTH** of that Node. In a tree, the total number of edges from root node to a leaf node in the longest path is said to be **Depth of the tree**. In simple words, the highest depth of any leaf node in a tree is said to be depth of that tree. In a tree, **depth of the root node is '0'**.



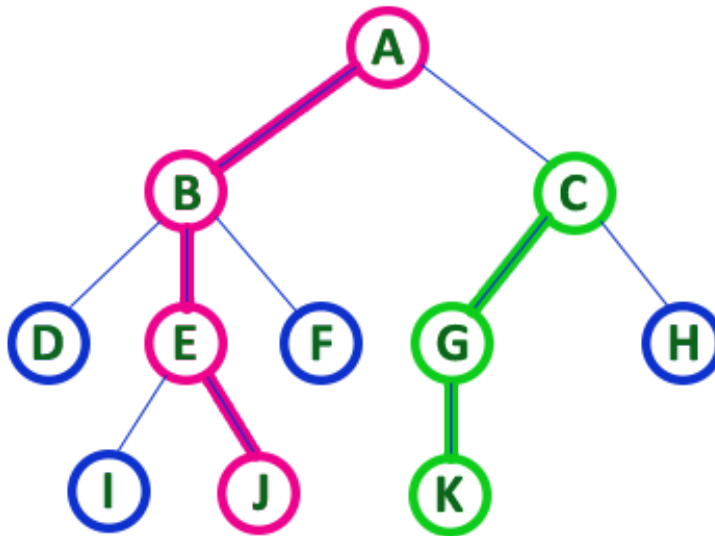
Here Depth of tree is 3

- In any tree, 'Depth of Node' is total number of Edges from root to that node.
- In any tree, 'Depth of Tree' is total number of edges from root to leaf in the longest path.

Tree Terminology

➤ Path

In a tree data structure, the sequence of Nodes and Edges from one node to another node is called as **PATH** between that two Nodes. Length of a Path is total number of nodes in that path. In below example the path **A - B - E - J** has length 4.



- In any tree, 'Path' is a sequence of nodes and edges between two nodes.

Here, 'Path' between A & J is

A - B - E - J

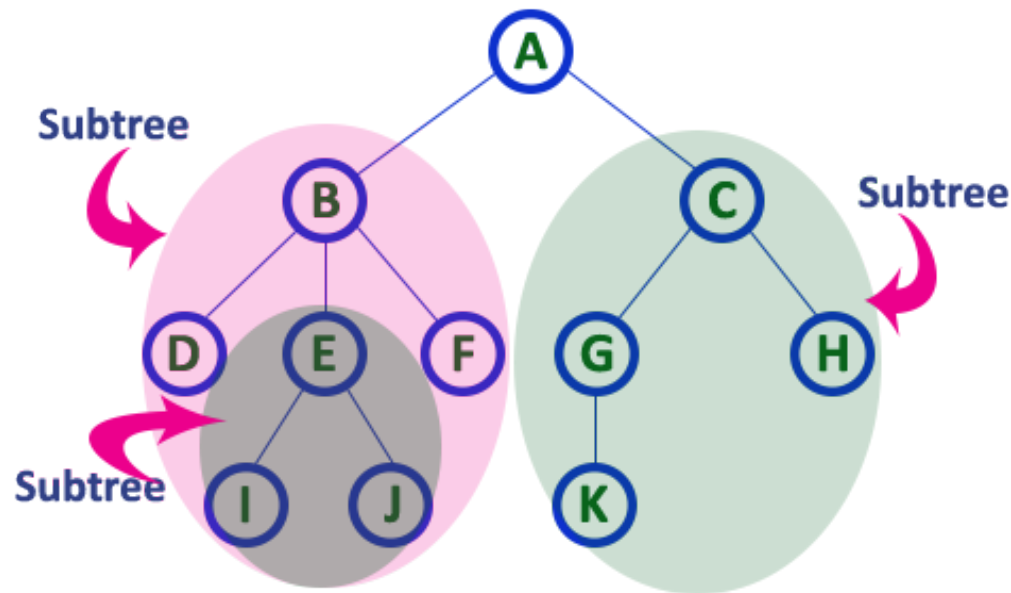
Here, 'Path' between C & K is

C - G - K

Tree Terminology

➤ Sub Tree

In a tree data structure, each child from a node forms a subtree recursively. Every child node will form a subtree on its parent node.



Tree Representations



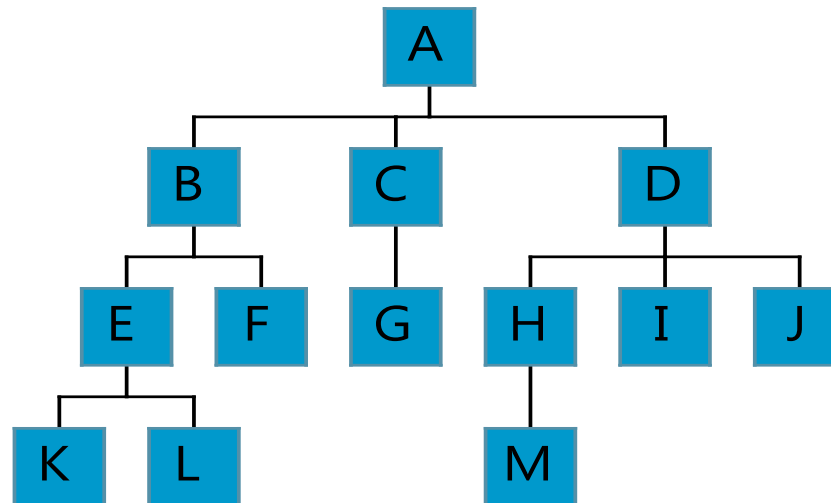
➤ A tree data structure can be represented in two methods.

➤ Those methods are as follows...

- **List Representation**
- **Left Child - Right Sibling Representation**
- **Representation as a degree-Two Tree**

Tree Representations

Consider the following tree...



List Representation

➤ In this representation, we use two types of nodes one for representing the node with data and another for representing only references. We start with a node with data from root node in the tree. Then it is linked to an internal node through a reference node and is linked to any other node directly. This process repeats for all the nodes in the tree.

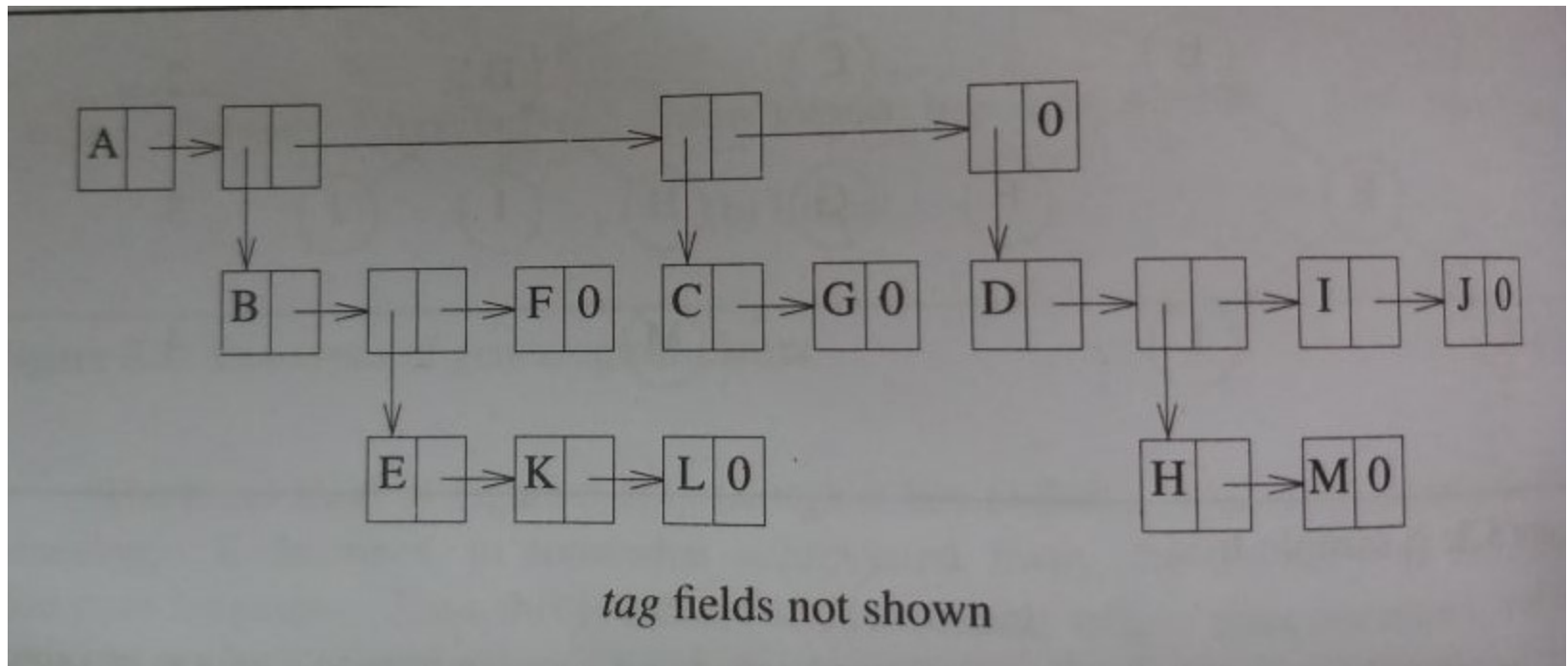
➤ The tree of figure could be written as the list

(A (B (E (K, L), F), C (G), D (H (M), I, J)))

data	link 1	link 2	...	link n
------	--------	--------	-----	--------

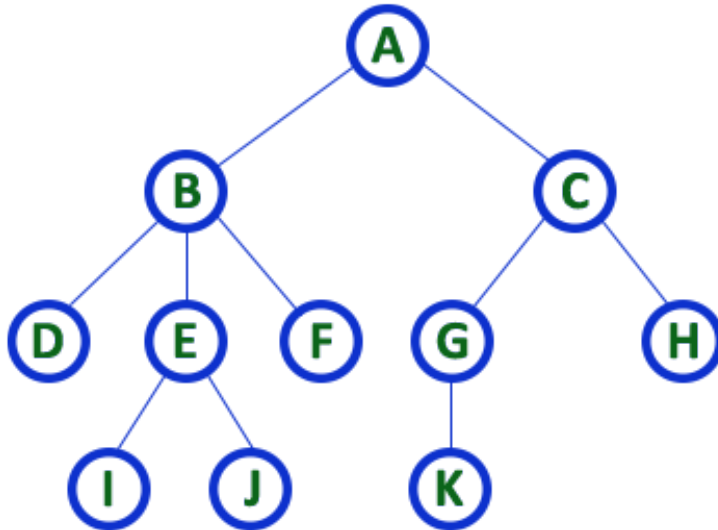
How many link fields are needed in such a representation?

List Representation



List Representation

➤ Consider following tree

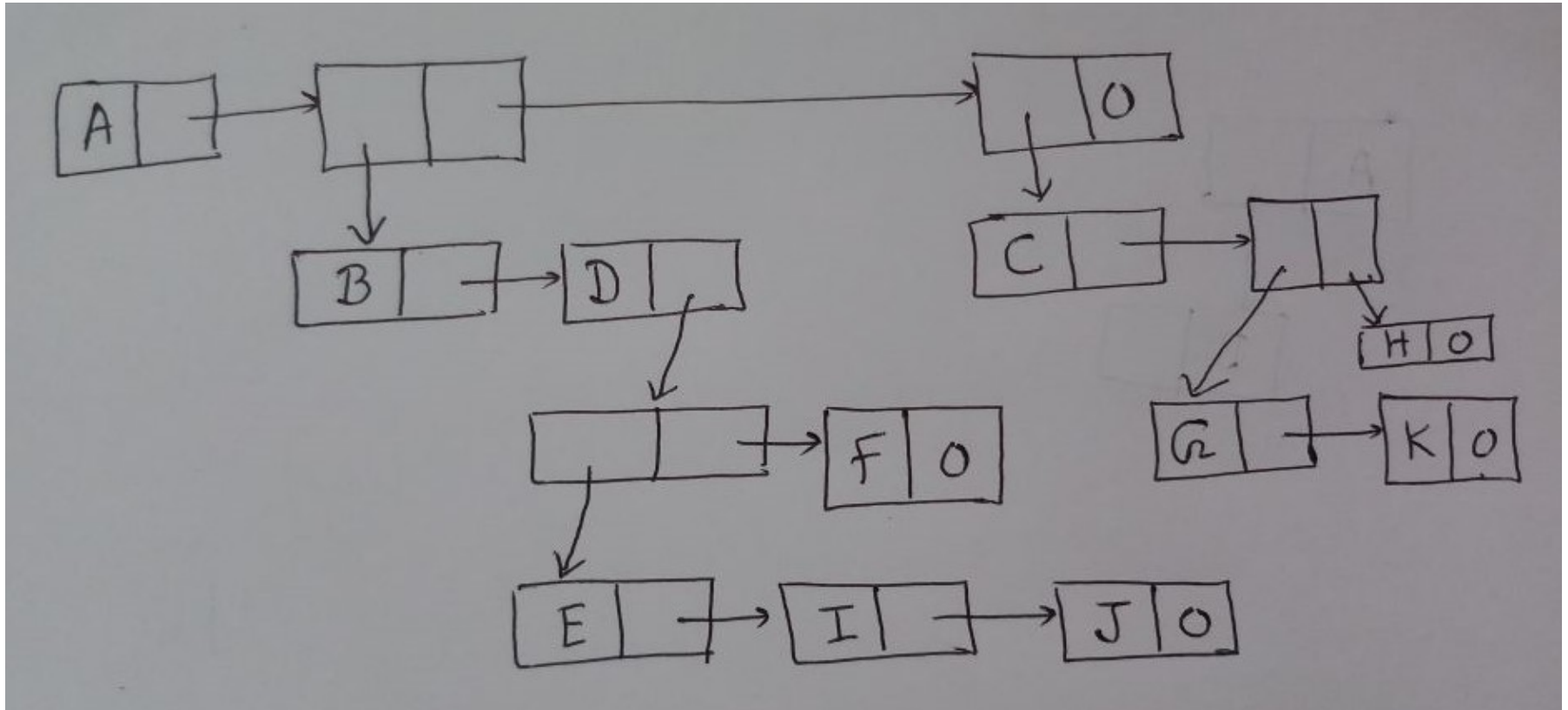


TREE with 11 nodes and 10 edges

- In any tree with '**N**' nodes there will be maximum of '**N-1**' edges
- In a tree every individual element is called as '**NODE**'

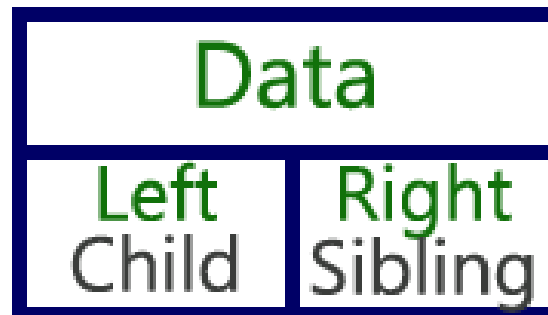
$(A(B(D,E(I,J),F),C(G(K),H)))$

List Representation



Left Child - Right Sibling Representation

- In this representation, we use list with one type of node which consists of three fields namely Data field, Left child reference field and Right sibling reference field.
- Data field stores the actual value of a node, left reference field stores the address of the left child and right reference field stores the address of the right sibling node.
- Graphical representation of that node is as follows...

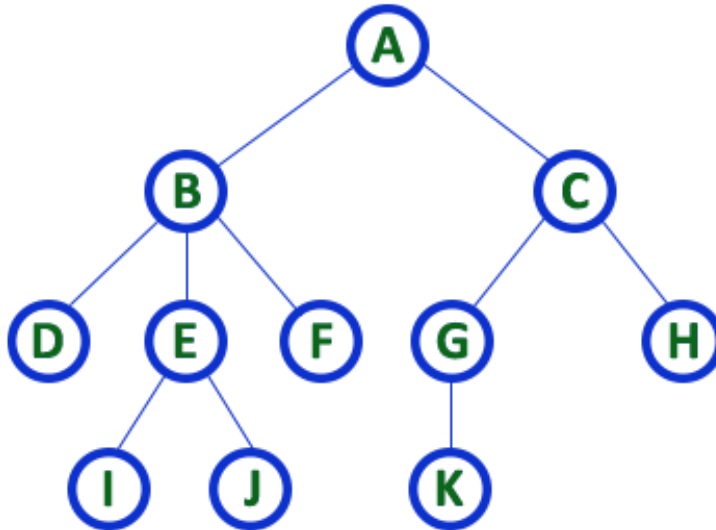


Left Child - Right Sibling Representation

- In this representation, every node's data field stores the actual value of that node.
 - If that node has left child, then left reference field stores the address of that left child node otherwise that field stores NULL.
 - If that node has right sibling then right reference field stores the address of right sibling node otherwise that field stores NULL.

Left Child - Right Sibling Representation

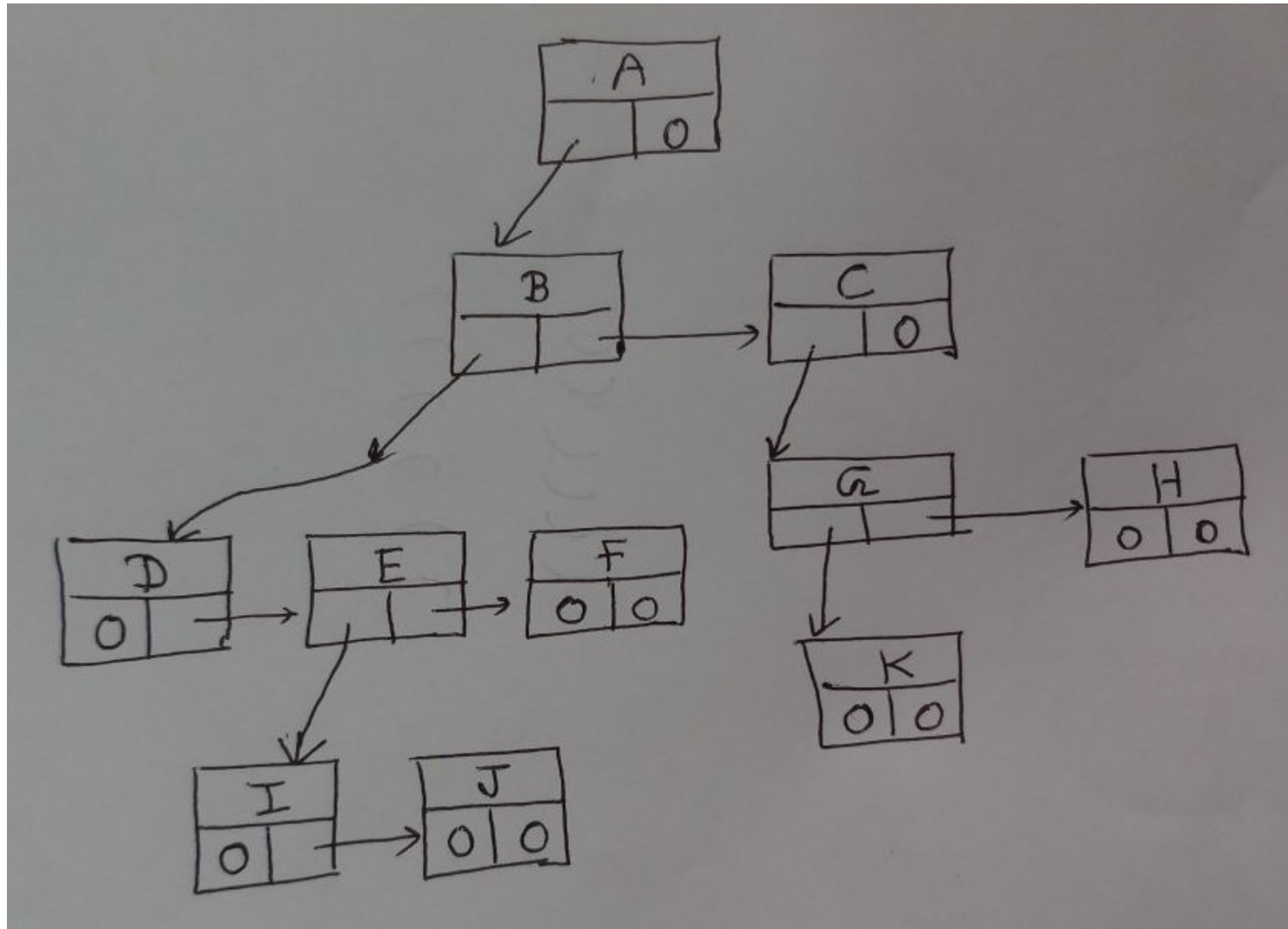
➤ Consider following tree



TREE with 11 nodes and 10 edges

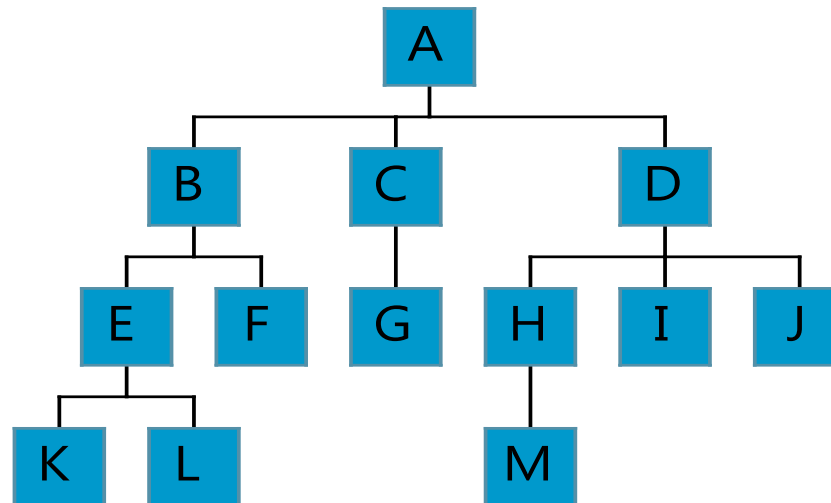
- In any tree with '**N**' nodes there will be maximum of '**N-1**' edges
- In a tree every individual element is called as '**NODE**'

Left Child - Right Sibling Representation

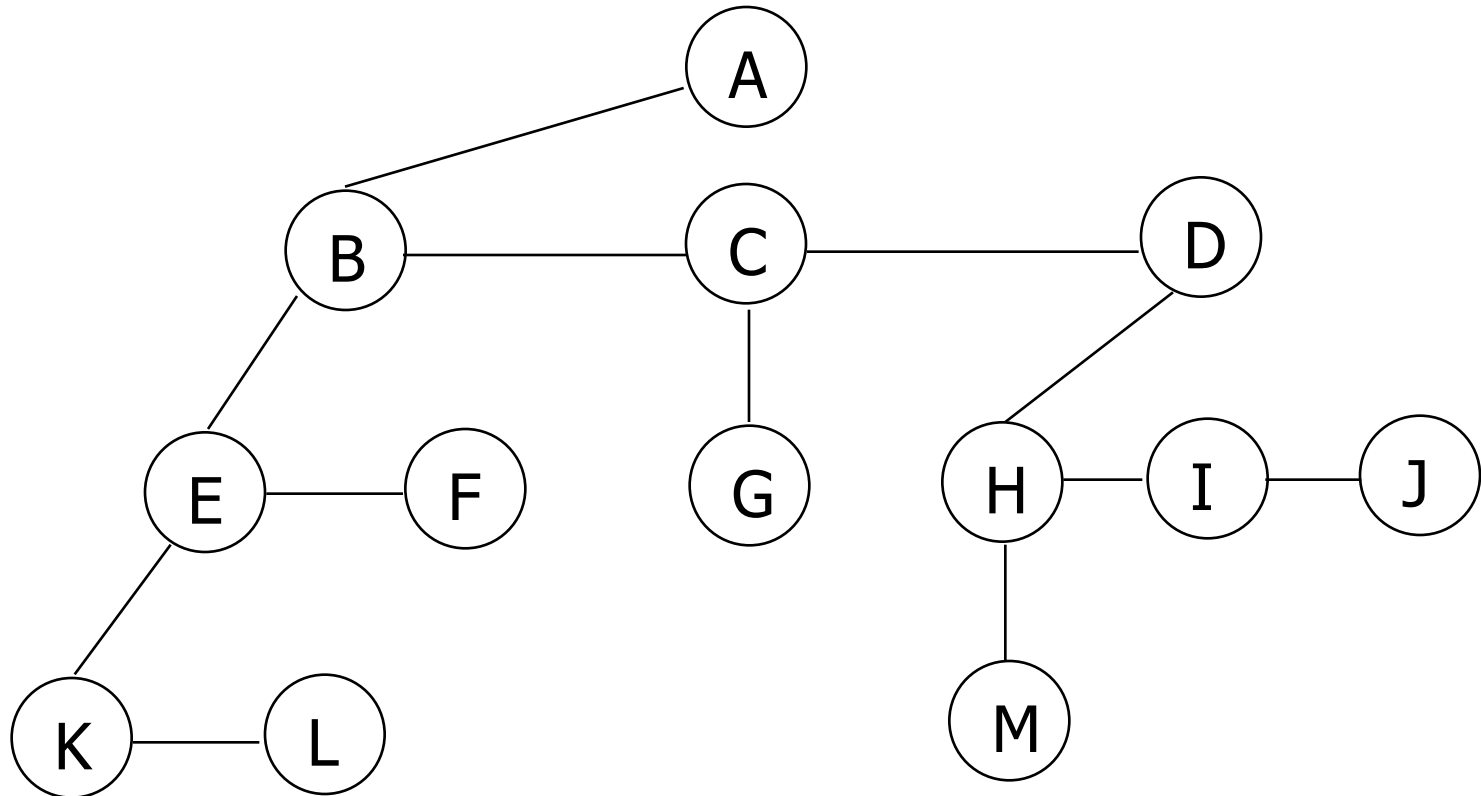


Left Child - Right Sibling Representation

Consider the following tree...

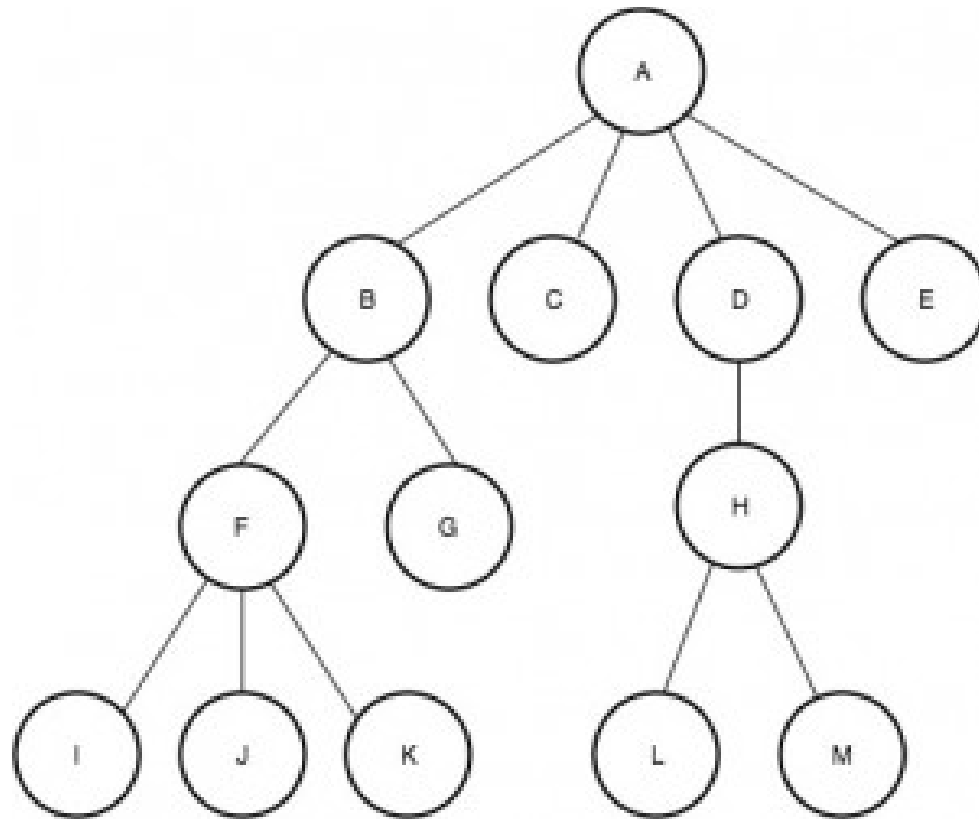


Left Child - Right Sibling Representation

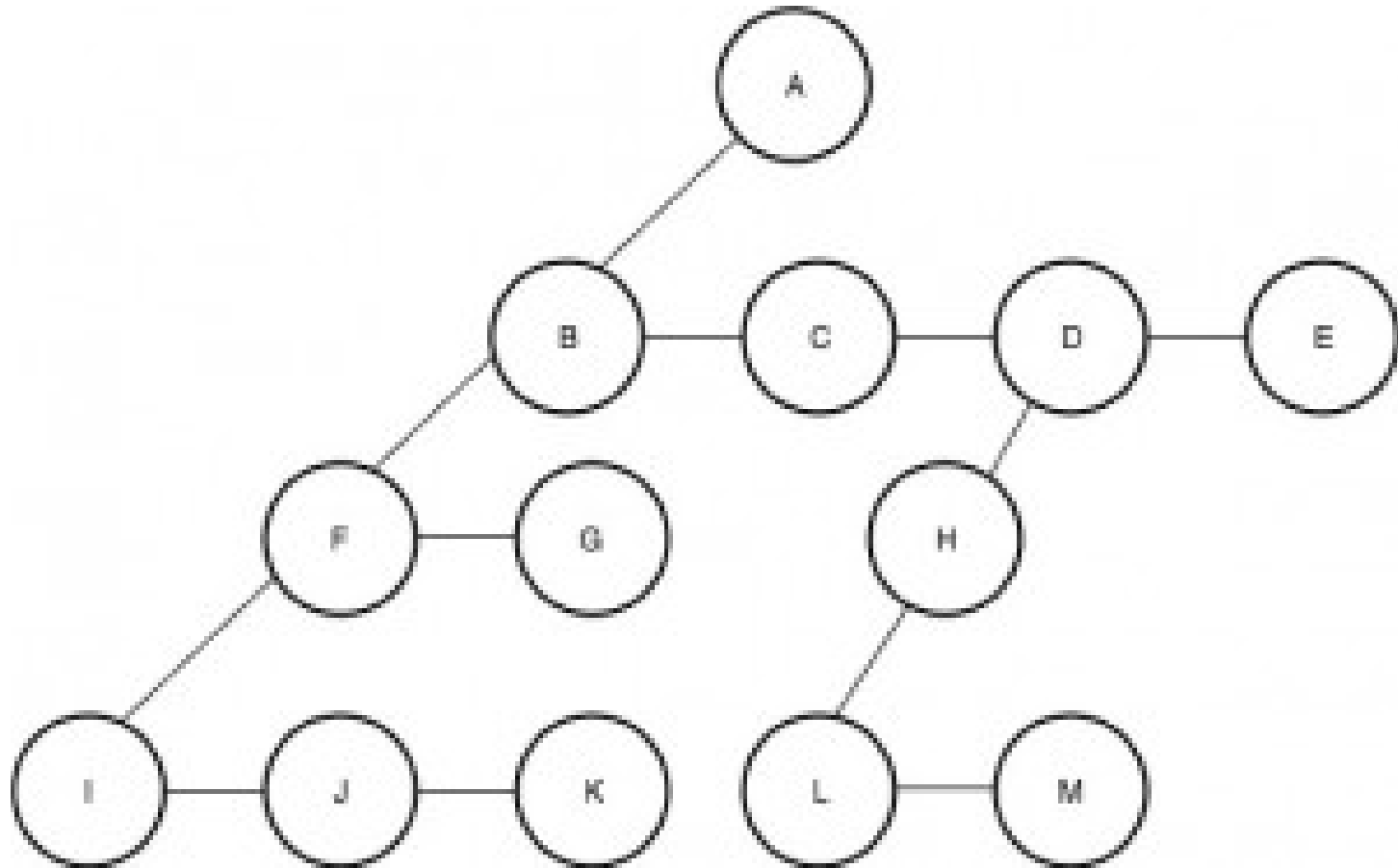


Left Child - Right Sibling Representation

Consider the following tree...



Left Child - Right Sibling Representation

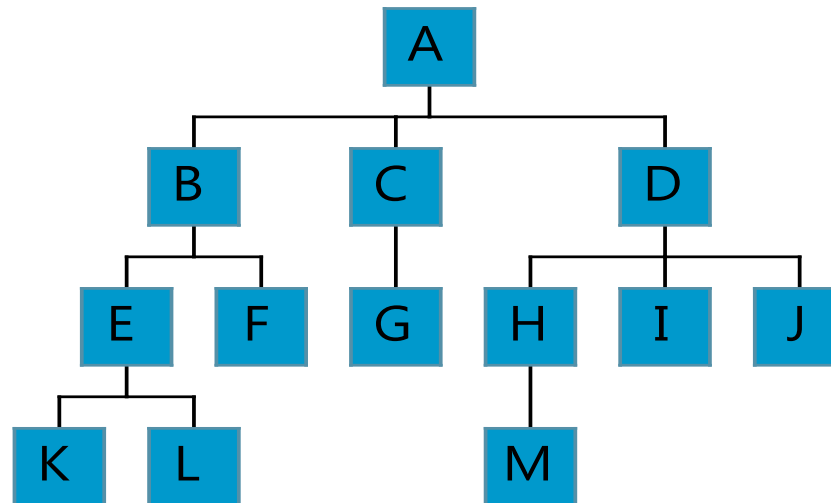


Representation as a Degree-Two Tree

- To obtain the degree-two tree representation of a tree, we simply rotate the right-sibling pointers in a left child-right sibling tree clockwise by 45 degrees.
- In the degree-two representation, we refer to the two children of a node as the left and right children.
- Notice that the right child of the root node of the tree is empty. This is always the case since the root of the tree we are transforming can never have a sibling.
- This type of representation are also known as binary tree.

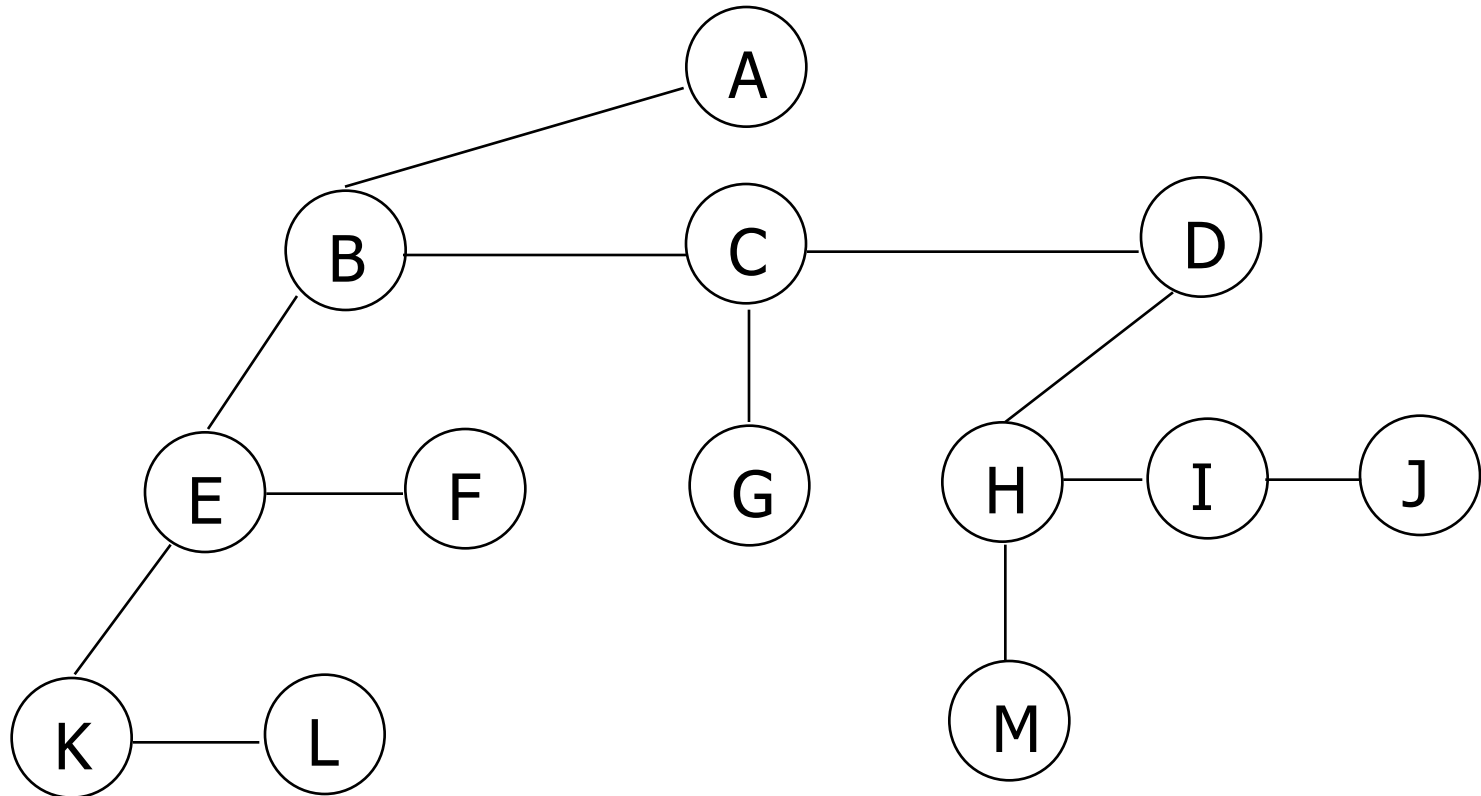
Representation as a Degree-Two Tree

Consider the following tree...



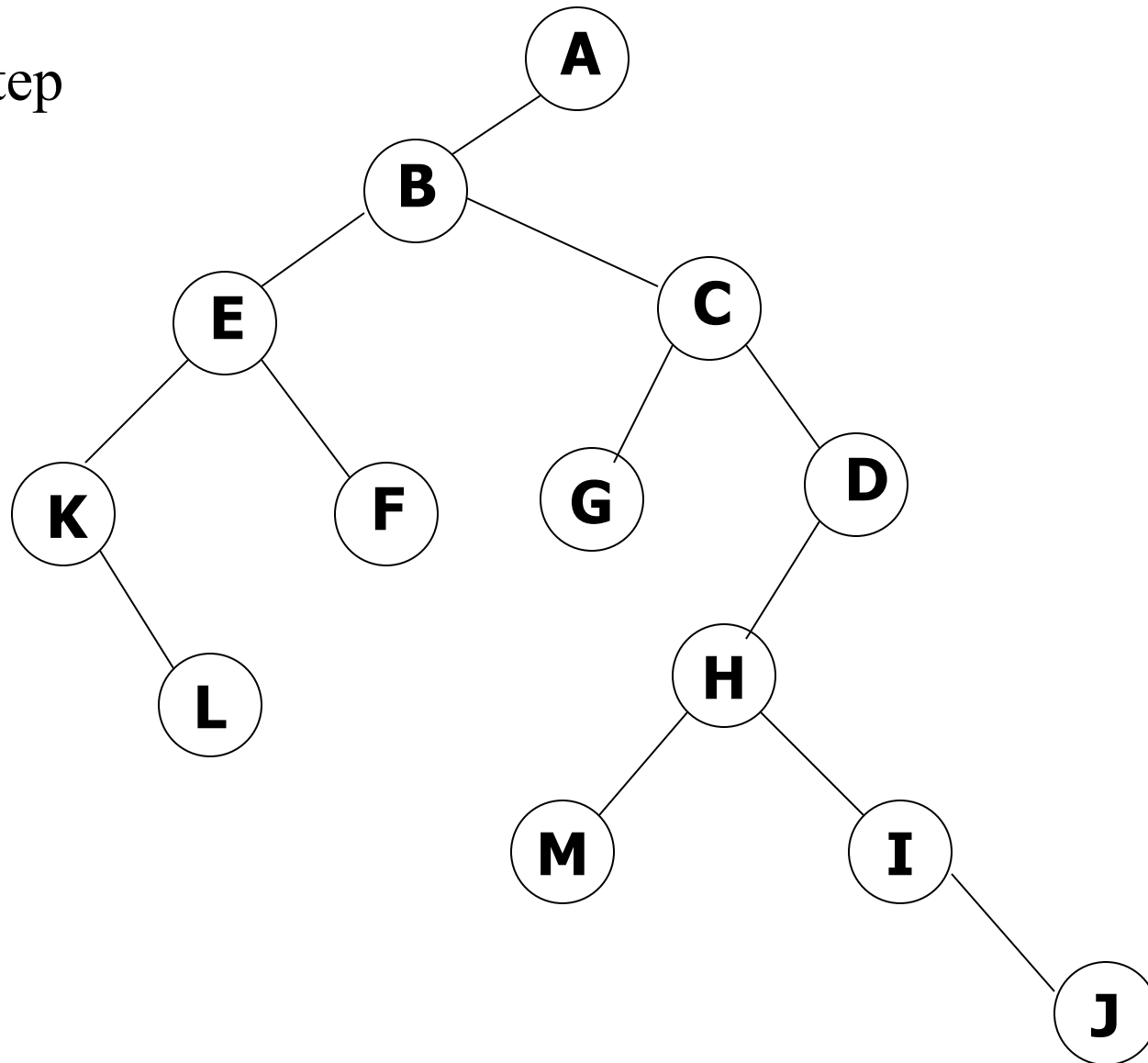
Representation as a Degree-Two Tree

➤ First Step

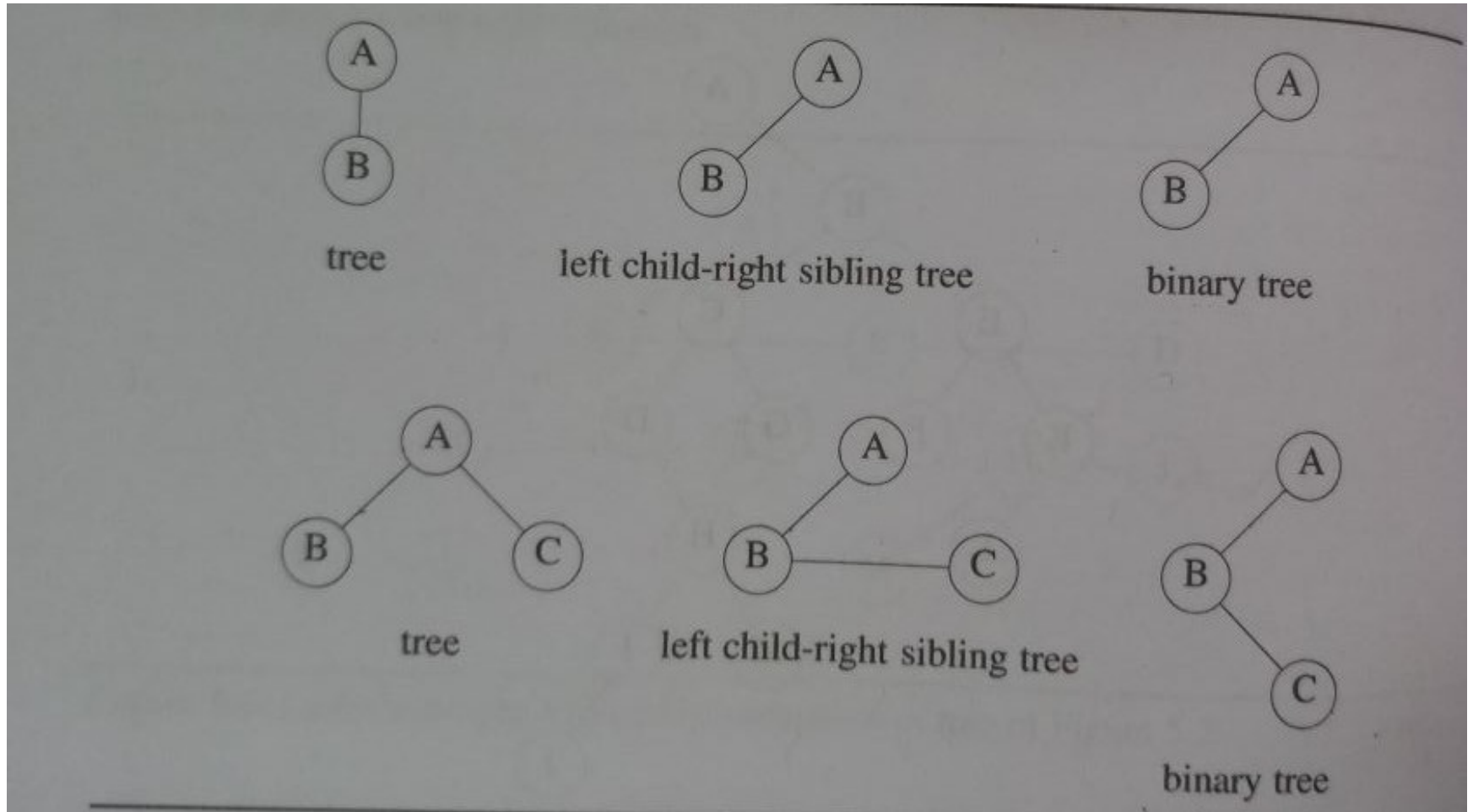


Representation as a Degree-Two Tree

➤ Final Step

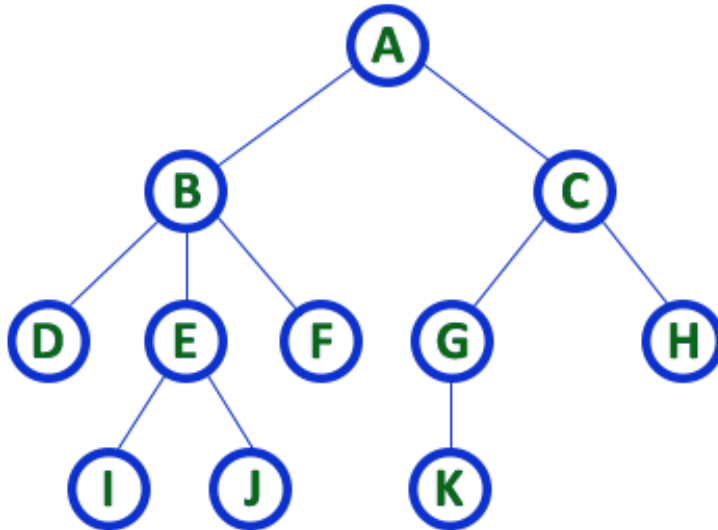


Representation as a Degree-Two Tree



Representation as a Degree-Two Tree

➤ Consider following tree

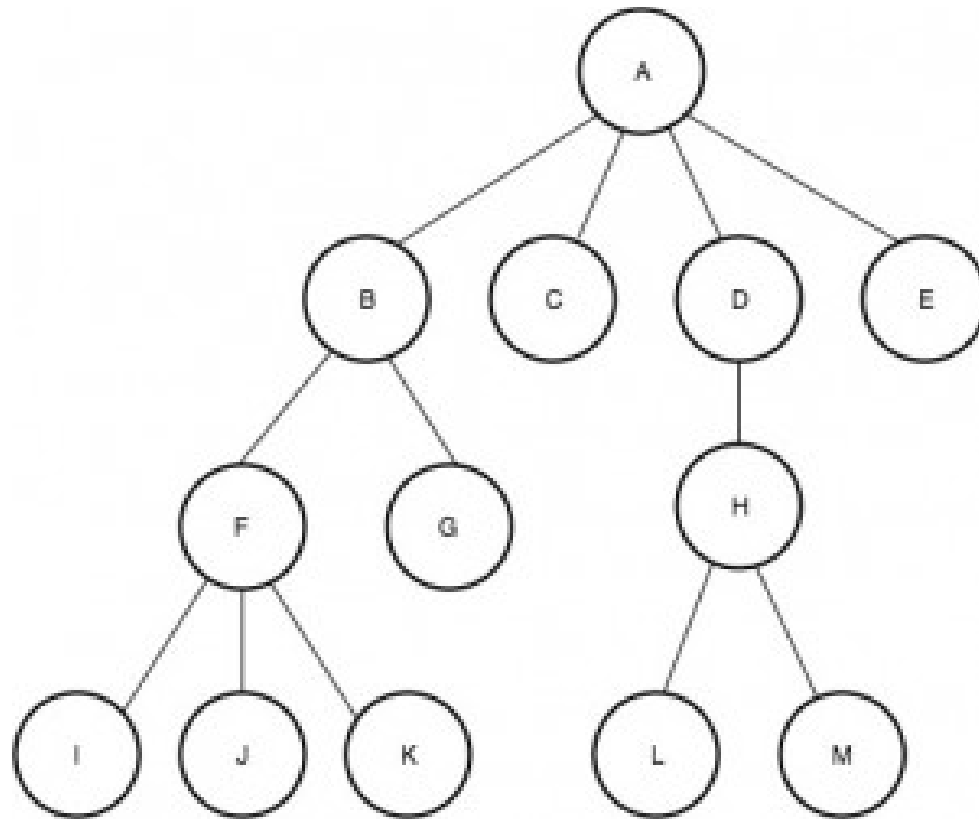


TREE with 11 nodes and 10 edges

- In any tree with '**N**' nodes there will be maximum of '**N-1**' edges
- In a tree every individual element is called as '**NODE**'

Representation as a Degree-Two Tree

Consider the following tree...



Binary Tree

- 
- We have seen that we can represent any tree as a binary tree.
 - In fact, binary trees are an important type of tree structure that occurs very often.
 - The main characteristics of a binary tree is the stipulation that the degree of any given node must not exceed two.

- 
- A binary tree is a finite set of nodes that is either empty or consists of a root and two disjoint binary trees called *the left subtree* and *the right subtree*.
 - Any tree can be transformed into binary tree.
 - by left child-right sibling representation
 - The left subtree and the right subtree are distinguished.

Structure of Binary Tree

- The structure defines only as minimal set of operations on binary trees which we use as a foundation on which to build additional operations-

structure *Binary_Tree* (abbreviated *BinTree*) is

objects: a finite set of nodes either empty or consisting of a root node, left *Binary_Tree*, and right *Binary_Tree*.

functions:

for all $bt, bt1, bt2 \in BinTree, item \in element$

<i>BinTree</i> Create()	::=	creates an empty binary tree
<i>Boolean</i> IsEmpty(<i>bt</i>)	::=	if (<i>bt</i> == empty binary tree) return <i>TRUE</i> else return <i>FALSE</i>
<i>BinTree</i> MakeBT(<i>bt1</i> , <i>item</i> , <i>bt2</i>)	::=	return a binary tree whose left subtree is <i>bt1</i> , whose right subtree is <i>bt2</i> , and whose root node contains the data <i>item</i> .
<i>BinTree</i> Lchild(<i>bt</i>)	::=	if (IsEmpty(<i>bt</i>)) return error else return the left subtree of <i>bt</i> .
<i>element</i> Data(<i>bt</i>)	::=	if (IsEmpty(<i>bt</i>)) return error else return the data in the root node of <i>bt</i> .
<i>BinTree</i> Rchild(<i>bt</i>)	::=	if (IsEmpty(<i>bt</i>)) return error else return the right subtree of <i>bt</i> .

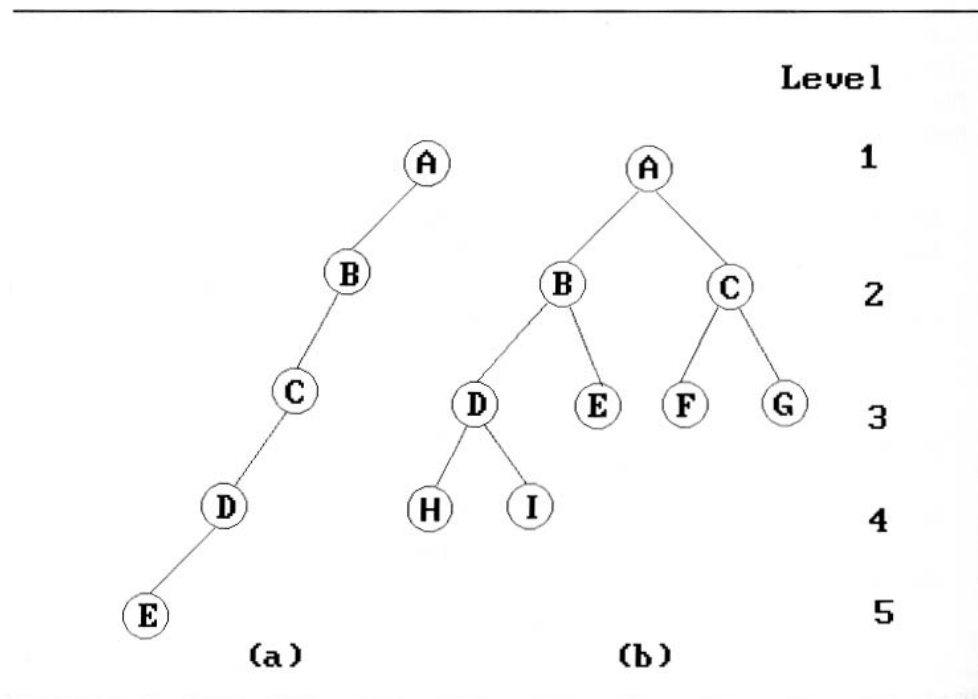
Structure 5.1: Abstract data type *Binary_Tree*

Type of Binary trees

➤ Two special kinds of binary trees:

(a) *skewed tree*, (b) *complete binary tree*

- The all leaf nodes of these trees are on two adjacent levels



Properties of Binary Trees

- **Lemma 5.1** [*Maximum number of nodes*]:
 1. The maximum number of nodes on **level i** of a binary tree is 2^{i-1} , $i \geq 1$.
 2. The maximum number of nodes in a **binary tree of depth k** is $2^k - 1$, $k \geq 1$.

Properties of Binary Trees

- **Lemma 5.2** [*Relation between number of leaf nodes and degree-2 nodes*]:
 - For any nonempty binary tree, T , if n_0 is the number of leaf nodes and n_2 is the number of nodes of degree 2, then $n_0 = n_2 + 1$.

Properties of Binary Trees

Proof

Let n_0 , n_1 , n_2 represent the nodes with no children, single child, and two children respectively.

$$n = n_0 + n_1 + n_2 \quad (1)$$

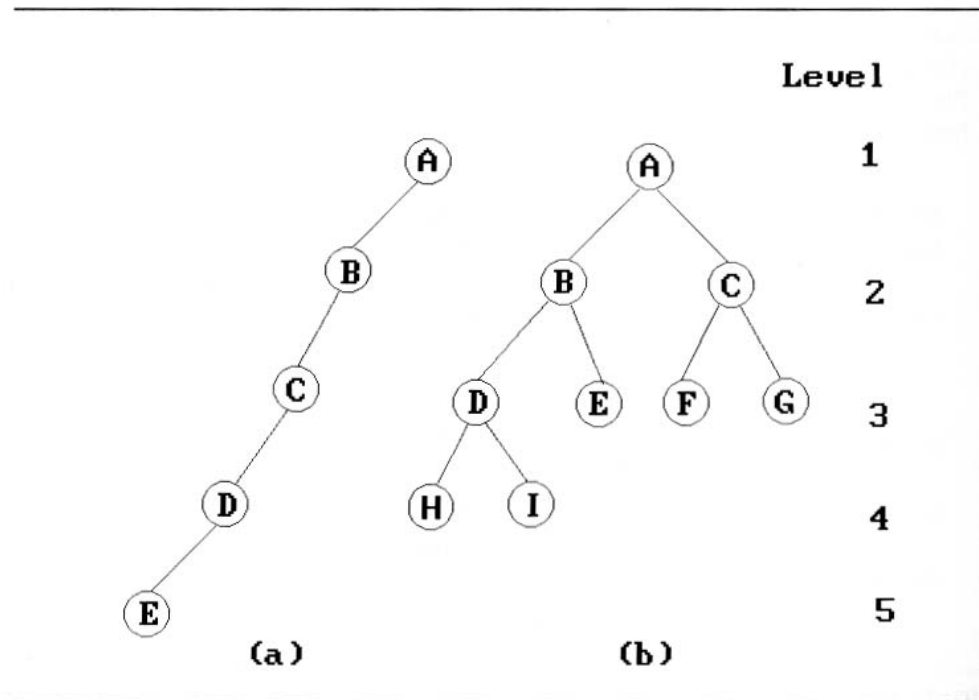
If we count the number of branches in a binary tree, we see that every node except the root has a branch leading into it. If B is the number of branches, then $n = B + 1$. All branches stem from a node of degree one or two. Thus, $B = n_1 + 2n_2$. Hence, we obtain

$$n = B + 1 = n_1 + 2n_2 + 1 \quad (2)$$

Subtracting Eq. (2) from Eq. (1), and rearranging terms, we get

$$n_0 = n_2 + 1$$

Properties of Binary Trees

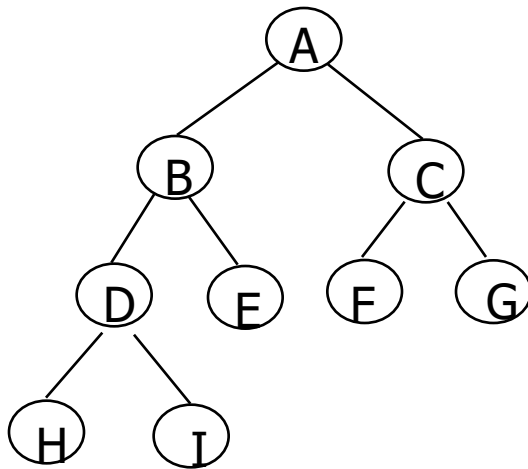


➤ In fig(a), $n_0=1$, $n_2=0$

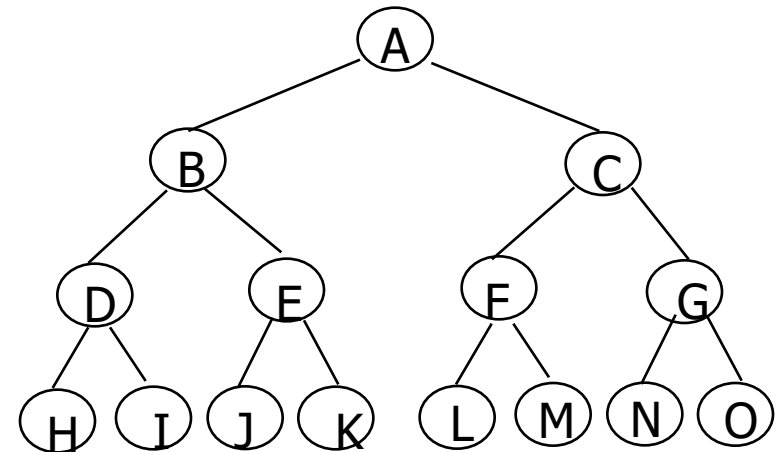
➤ In gif(b), $n_0=5$, $n_2=4$

Full BT VS Complete BT

- A full binary tree of depth k is a binary tree of depth k having $2^k - 1$ nodes, $k \geq 0$.
- A binary tree with n nodes and depth k is complete *iff* its nodes correspond to the nodes numbered from 1 to n in the full binary tree of depth k .



Complete binary tree



Full binary tree of depth 4

Binary Tree Representations



➤ A binary tree data structure is represented using two methods.

Those methods are as follows...

- **Array Representation**
- **Linked List Representation**

Binary Tree Representation using Array

- In array representation of binary tree, we use a one dimensional array (1-D Array) to represent a binary tree.
- To represent a binary tree of depth '**n**' using array representation, we need one dimensional array with a maximum size of $2^{n+1} - 1$.

Binary Tree Representation using Array

Binary tree representations (using array)

- **Lemma 5.3:** If a complete binary tree with n nodes is represented sequentially, then for any node with index i , $1 \leq i \leq n$, we have

1. $\text{parent}(i)$ is at $\lfloor i/2 \rfloor$ if $i \neq 1$.

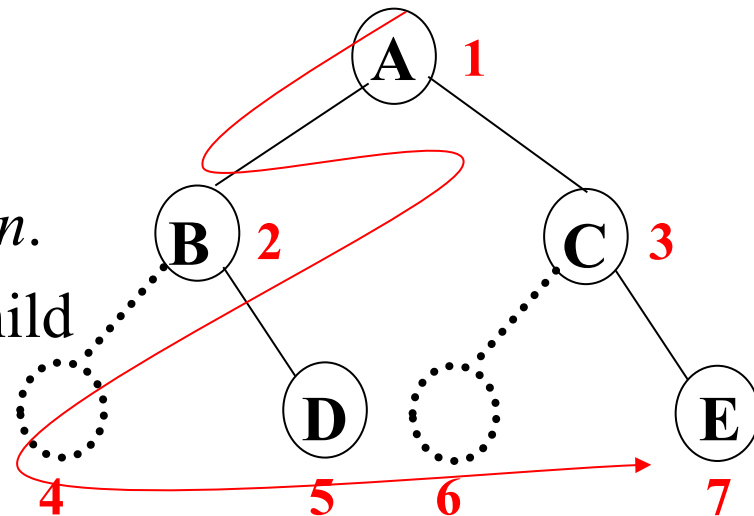
If $i = 1$, i is at the root and has no parent.

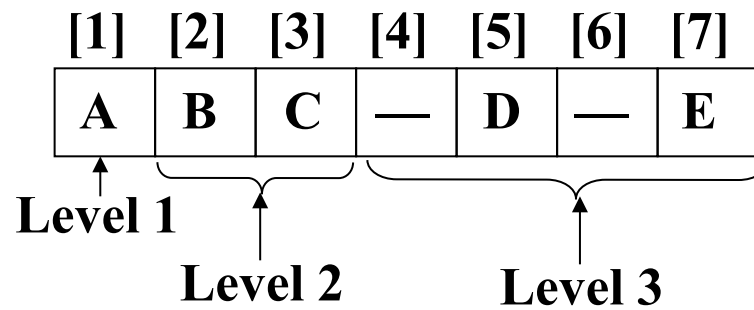
2. $\text{LeftChild}(i)$ is at $2i$ if $2i \leq n$.

If $2i > n$, then i has no left child.

3. $\text{RightChild}(i)$ is at $2i+1$ if $2i+1 \leq n$.

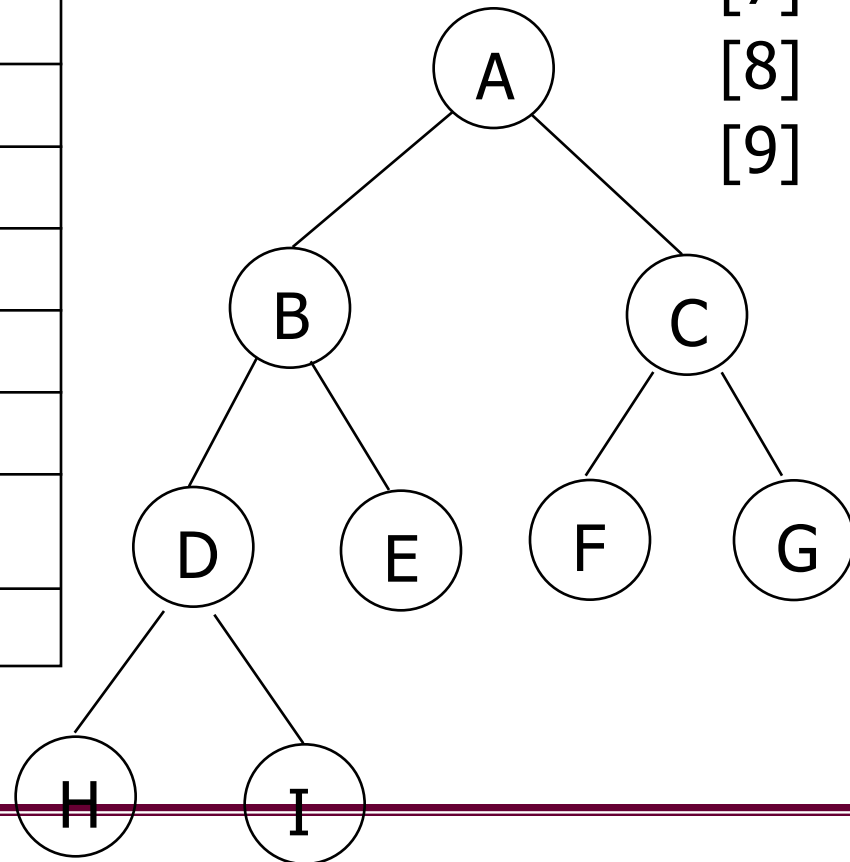
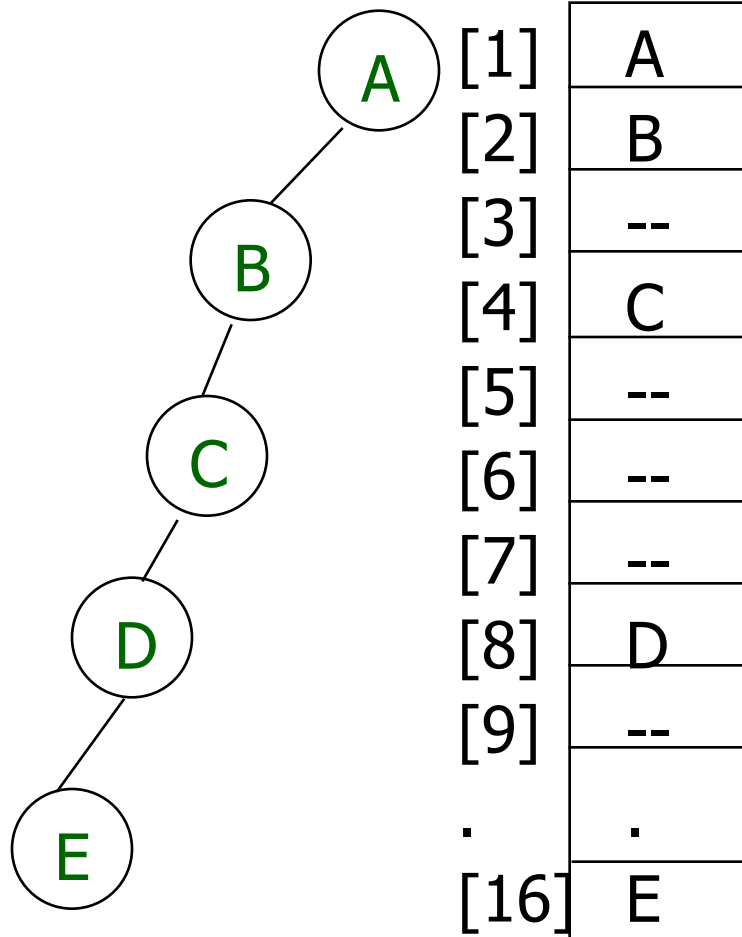
If $2i + 1 > n$, then i has no right child





[1]	A
[2]	B
[3]	C
[4]	D
[5]	E
[6]	F
[7]	G
[8]	H
[9]	I

(1) waste space
(2) insertion/deletion problem

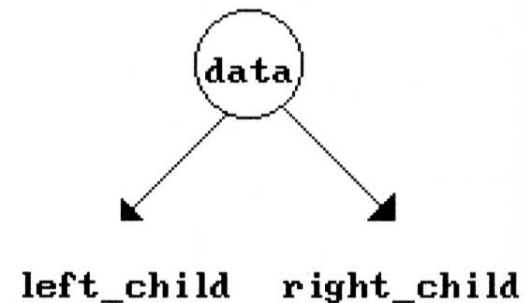
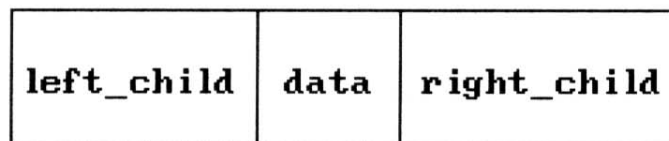


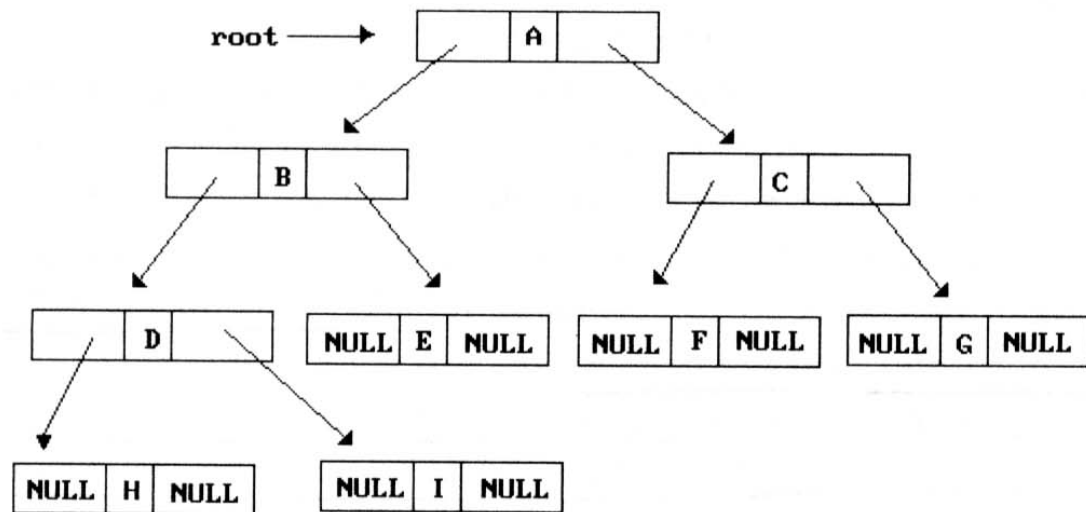
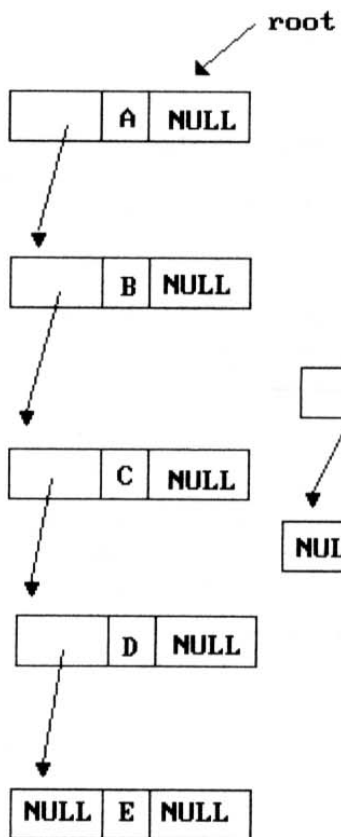
- 
- Waste spaces: in the worst case, a skewed tree of depth k requires $2^k - 1$ spaces. Of these, *only k spaces will be occupied*
 - Insertion or deletion of nodes from the middle of a tree requires the *movement of potentially many nodes* to reflect the *change in the level* of these nodes

Binary Tree Representation using Linked List

- Although the array representation is good for complete binary trees, it is wasteful for many other binary trees. In addition, the representation suffers from the general inadequacies of sequential representations.
- Insertion and deletion of nodes from the middle of a tree require the movement of potentially many nodes to reflect the change in level number of these nodes.
- These problems can be overcome easily through the use of linked representation.
- Each node has three fields: left child, data, and right child.

```
typedef struct node *tree_pointer;
typedef struct node {
    int data;
    tree_pointer left_child, right_child;
};
```





Binary Tree Traversal

- When we wanted to display a binary tree, we need to follow some order in which all the nodes of that binary tree must be displayed. In any binary tree displaying order of nodes depends on the traversal method.
- **Displaying (or) visiting order of nodes in a binary tree is called as Binary Tree Traversal.**
- Traversal is a process to visit all the nodes of a tree and may print their values too. Because, all nodes are connected via edges (links) we always start from the root (head) node. That is, we cannot randomly access a node in a tree.



➤ There are three types of binary tree traversals.

- **In - Order Traversal (LDR)**
- **Pre - Order Traversal (DLR)**
- **Post - Order Traversal (LRD)**

➤ Three other types of binary tree traversals

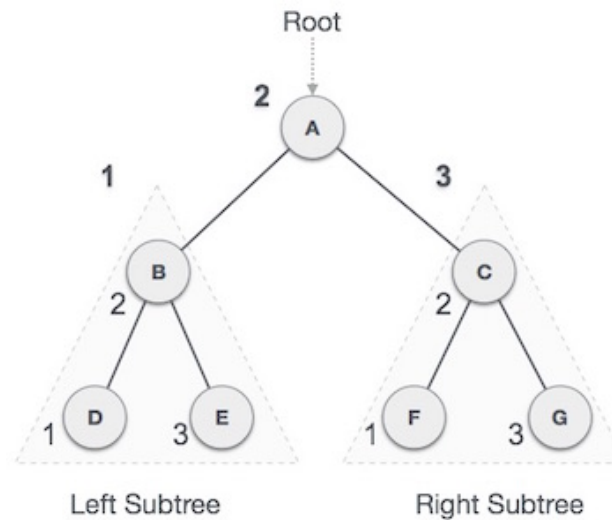
- Reverse In-Order Traversal (RDL)
- Reverse Pre-Order Traversal (DRL)
- Reverse Post-Order Traversal (RLD)

In-order Traversal

- In In-Order traversal, the root node is visited between left child and right child. In this traversal, the left child node is visited first, then the root node is visited and later we go for visiting right child node. This in-order traversal is applicable for every root node of all subtrees in the tree. This is performed recursively for all nodes in the tree.
- If a binary tree is traversed **in-order**, the output will produce sorted key values in an ascending order.
- **Algorithm**
 - Until all nodes are traversed –
 - **Step 1** – Recursively traverse left subtree.
 - **Step 2** – Visit root node.
 - **Step 3** – Recursively traverse right subtree.



```
void inorder(tree_pointer ptr)
/* inorder tree traversal */
{
    if (ptr) {
        inorder(ptr->left_child);
        printf("%d", ptr->data);
        inorder(ptr->right_child);
    }
}
```



➤ We start from **A**, and following in-order traversal, we move to its left subtree **B**. **B** is also traversed in-order. The process goes on until all the nodes are visited. The output of inorder traversal of this tree will be –

$$\mathbf{D \rightarrow B \rightarrow E \rightarrow A \rightarrow F \rightarrow C \rightarrow G}$$

Pre-Order Traversal

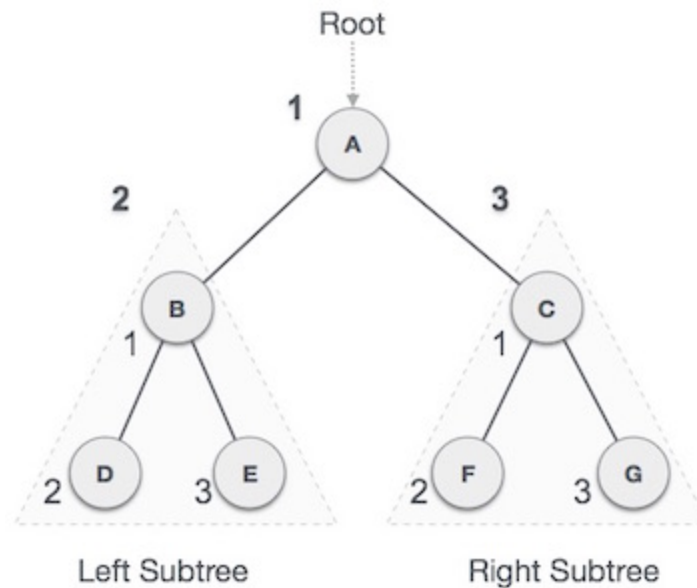
➤ In Pre-Order traversal, the root node is visited before left child and right child nodes. In this traversal, the root node is visited first, then its left child and later its right child. This pre-order traversal is applicable for every root node of all subtrees in the tree.

➤ Algorithm

- Until all nodes are traversed –
- **Step 1** – Visit root node.
- **Step 2** – Recursively traverse left subtree.
- **Step 3** – Recursively traverse right subtree.



```
void preorder(tree_pointer ptr)
/* preorder tree traversal */
{
    if (ptr) {
        printf("%d", ptr->data);
        preorder(ptr->left-child);
        preorder(ptr->right-child);
    }
}
```



➤ We start from **A**, and following pre-order traversal, we first visit **A** itself and then move to its left subtree **B**. **B** is also traversed pre-order. The process goes on until all the nodes are visited. The output of pre-order traversal of this tree will be –

$A \rightarrow B \rightarrow D \rightarrow E \rightarrow C \rightarrow F \rightarrow G$

Post-Order Traversal

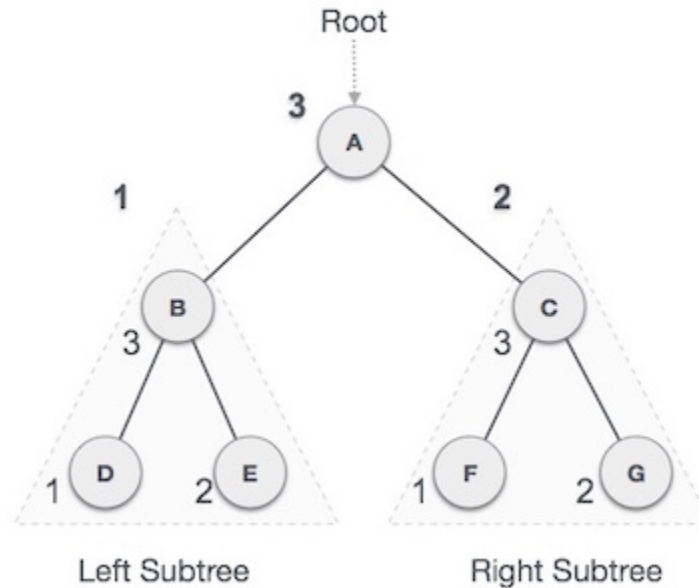
➤ In this traversal method, the root node is visited last, hence the name. First we traverse the left subtree, then the right subtree and finally the root node.

➤ Algorithm

- Until all nodes are traversed –
- **Step 1** – Recursively traverse left subtree.
- **Step 2** – Recursively traverse right subtree.
- **Step 3** – Visit root node.



```
void postorder(tree_pointer ptr)
/* postorder tree traversal */
{
    if (ptr) {
        postorder(ptr->left_child);
        postorder(ptr->right_child);
        printf("%d", ptr->data);
    }
}
```



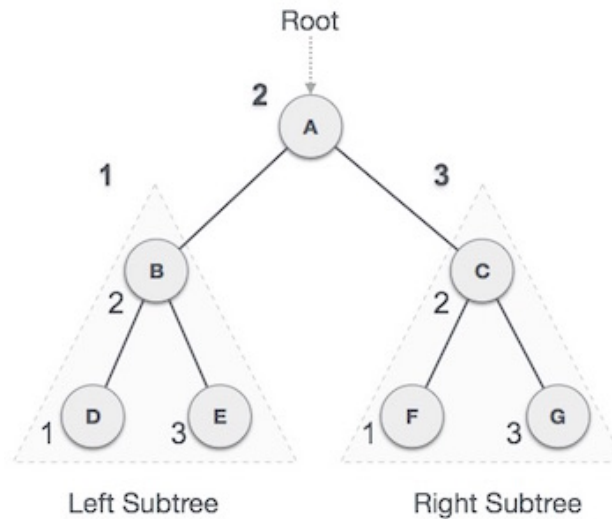
➤ We start from **A**, and following Post-order traversal, we first visit the left subtree **B**. **B** is also traversed post-order. The process goes on until all the nodes are visited. The output of post-order traversal of this tree will be –

$D \rightarrow E \rightarrow B \rightarrow F \rightarrow G \rightarrow C \rightarrow A$



➤ Reverse Tree Traversal

- Reverse In-Order Traversal (RDL)
- Reverse Pre-Order Traversal (DRL)
- Reverse Post-Order Traversal (RLD)



➤ Reverse In-Order Traversal

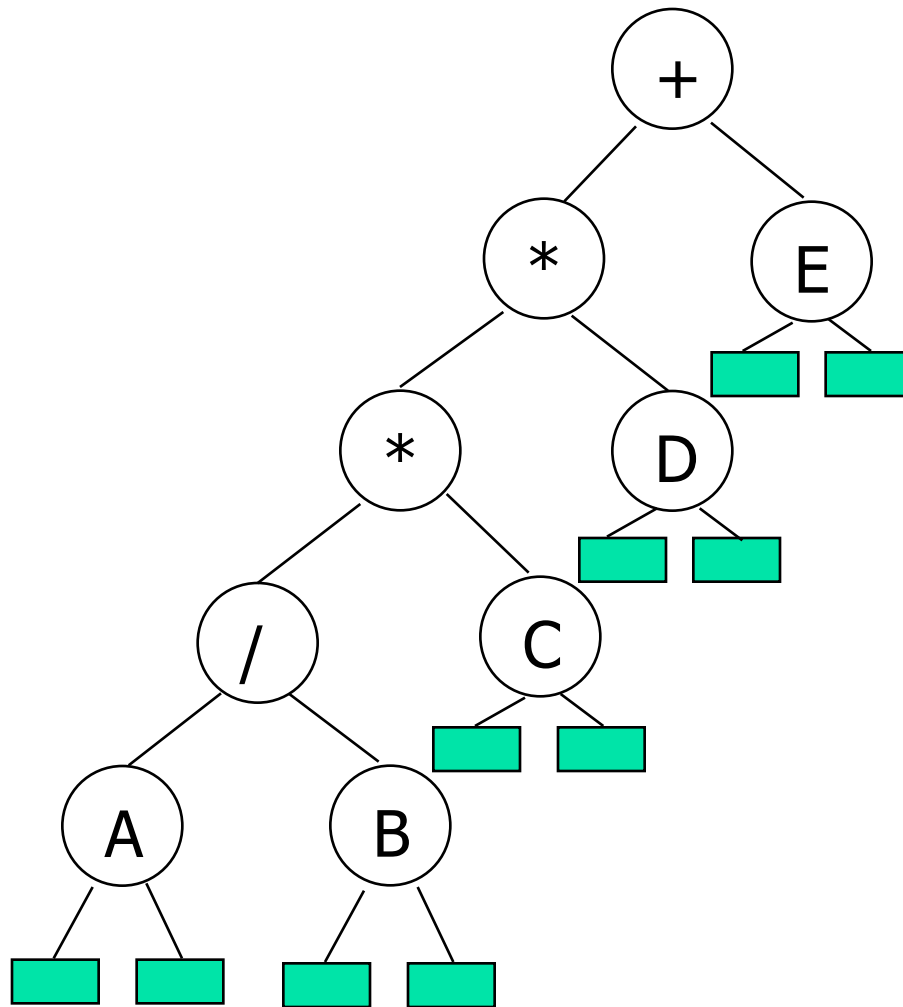
G-C-F-A-E-B-D

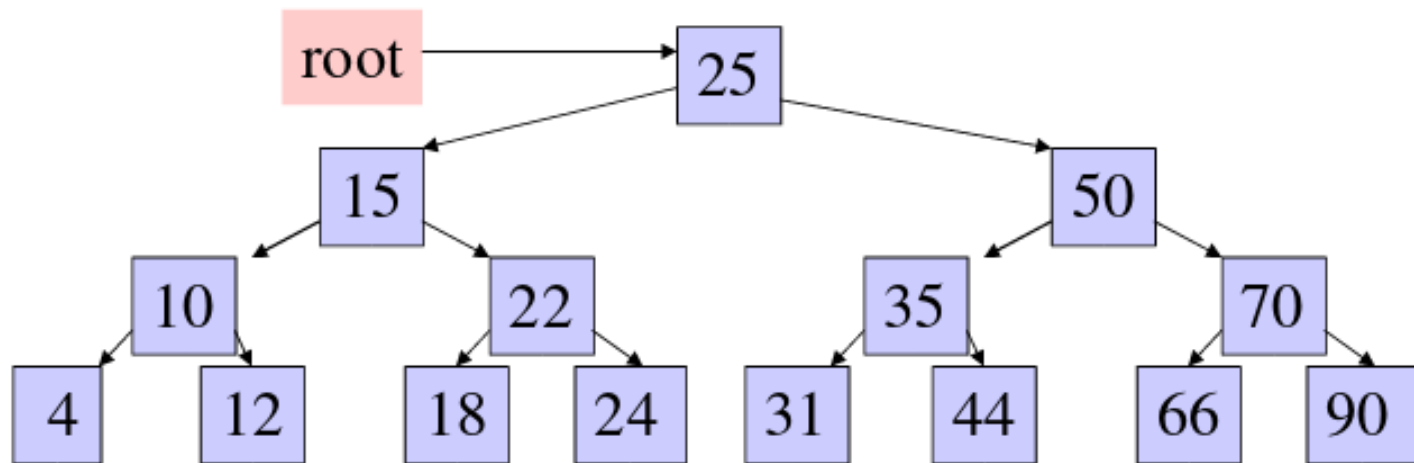
➤ Reverse Pre-Order Traversal

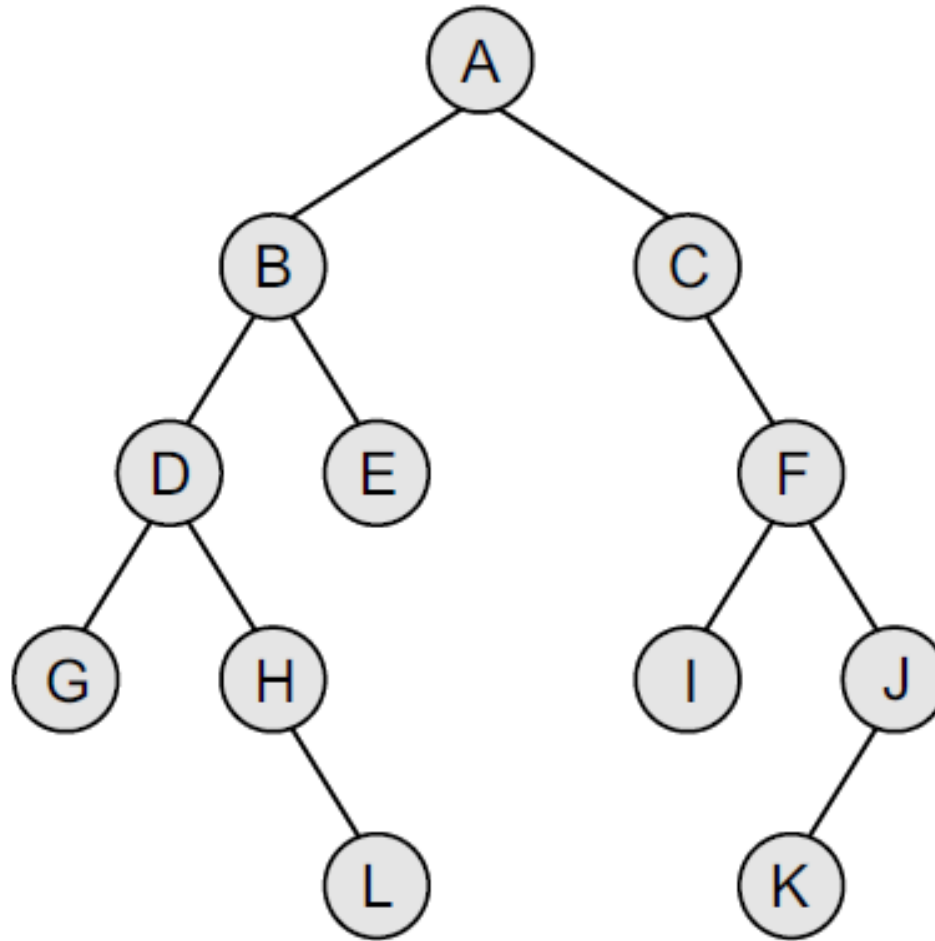
A-C-G-F-B-E-D

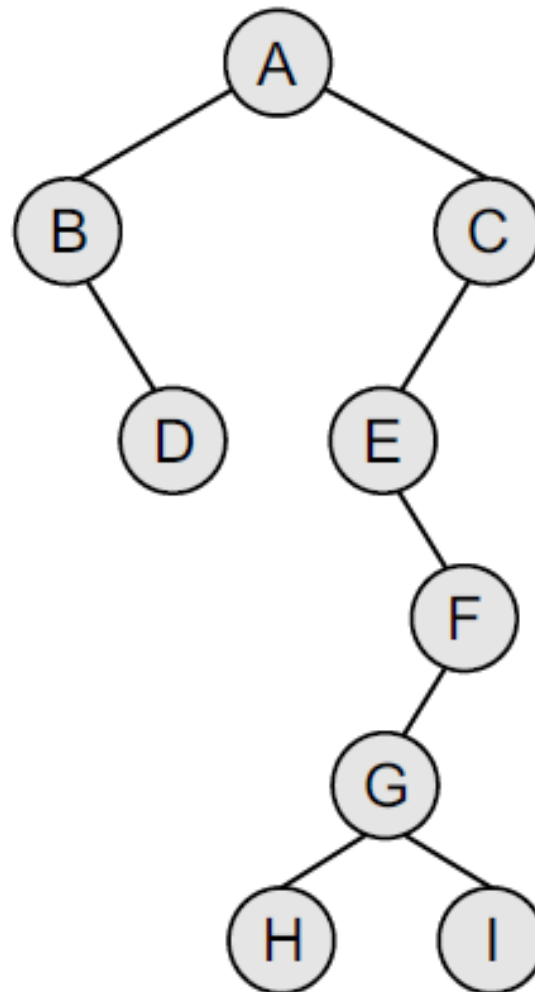
➤ Reverse Post-Order Traversal

G-F-C-E-D-B-A









Level-Order Traversal



➤ Level-order traversal

- method:
 - We visit the root first, then the root's left child, followed by the root's right child.
 - We continue in this manner, visiting the nodes at each new level from the leftmost node to the rightmost nodes
- This traversal requires a queue to implement

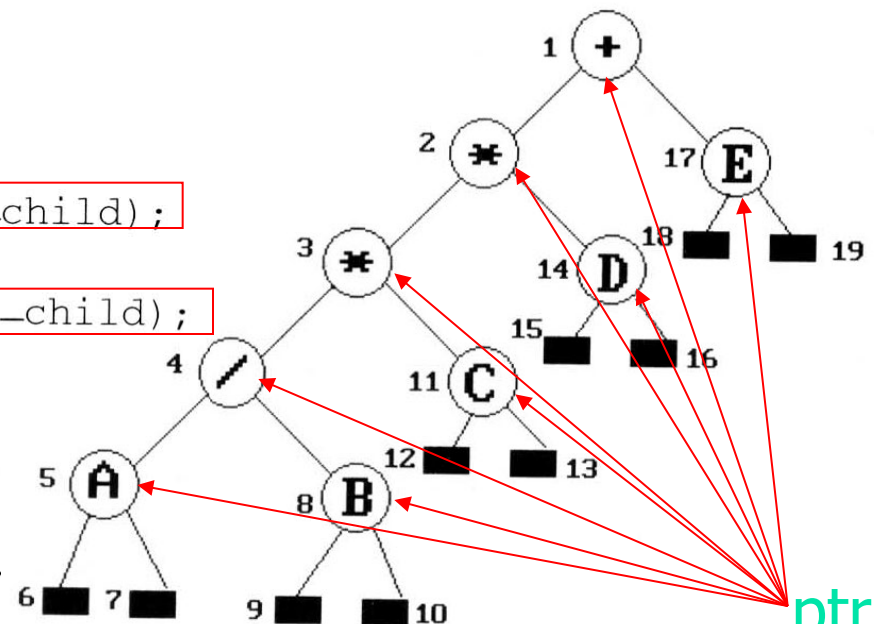
➤ Level-order traversal (using queue)

```
void level_order(tree_pointer ptr)
/* level order tree traversal */
{
    int front = rear = 0;
    tree_pointer queue[MAX_QUEUE_SIZE];
    if (!ptr) return; /* empty tree */
    addq(front, &rear, ptr);
    for (;;) {
        ptr = deleteq(&front, rear);
        if (ptr) {
            printf("%d", ptr->data);
            if (ptr->left_child)
                addq(front, &rear, ptr->left_child);
            if (ptr->right_child)
                addq(front, &rear, ptr->right_child);
        }
        else break;
    }
}
```

FIFO

output: $+ * E * D / C A B$

2	17	3	14	4	11	5	8
*	E	*	D	/	C	A	B



Program 5.5: Level order traversal of a binary tree



Constructing a Binary Tree from Traversal Results